***Abstract*—Enterprise AI governance frameworks — NIST AI RMF, ISO/IEC 42001, and supervisory guidance such as OSFI Guideline E-23 — specify governance, assessment, treatment, monitoring, and accountability requirements, but do not prescribe a deterministic, traceable transformation from approved risk artifacts into the machine-evaluable predicates enforced at an execution boundary; deterministic runtime-enforcement architectures begin at a pre-approved predicate set. The translation between them is, in practice, undocumented, producing silent narrowing, orphan controls, and heat-map scores misused as authorization criteria. This paper formalizes the translation as two separated stages. A governed semantic refinement Ψ_K — a relation over the approved assessment, closed into a single-valued mapping only by a signed judgment record J — converts each assessed risk, under a versioned Control Derivation Catalog, into signed Approved Control Specifications and assigns every risk exactly one disposition (runtime, non-runtime, accepted, or unresolved), so that no risk prose is silently treated as executable semantics. A deterministic compiler Φ then maps approved specifications onto a canonically serialized (RFC 8785), signed, immutable predicate bundle closed against an approved action-authority matrix, with lifecycle state carried in a separately signed registry and every predicate carrying an explicit enforcement phase. Authorization is scoped to discrete, latency-bounded transitions and resolves to PERMIT, DENY (an evaluated prohibition), or HOLD (a required condition unresolved), the latter two non-actuating; control-loop-rate hazards are routed to an independent runtime safety layer with standing authority, and permits are bound to a digest of declared world state. A consequence-class gate coverage rule over a domain-parameterized profile C^* is formalized with a gate representability theorem — exact conditions under which a mandatory-gate assignment is a function of the likelihood $\times$ impact score at all, or of a threshold on it — and three propositions: monotone-threshold non-separability, score-collision non-representability — a scalar risk score erases the consequence information the approved gate assignment requires — and qualified gate invariance under rating changes. A temporally valid traceability model binds obligation $\rightarrow$ disposition $\rightarrow$ specification $\rightarrow$ predicate $\rightarrow$ receipt, with authorization-before-commit-before-actuation ordering and a standing-origin carve-out for safety-layer interventions. Three worked cases — an investment-research agent, a transaction-authorization agent whose upstream chain to a previously evaluated predicate family is made explicit, and a fully specified supervised-autonomy surgical design case anchored to ISO 14971, IEC 62304, and IEC 80601-2-77 — and a preregistered evaluation plan operationalize the claims. On the two pinned financial artifacts the method achieves total disposition, zero orphan predicates, complete traceability, full consequence-class coverage, and translation determinism of 1.000 over 31 compilation runs per case, reproduced across 8 independently installed environments on three operating systems; gate divergence against the enterprise's predeclared heat-map rule is GD(15) = 3 with $\mathrm{GD}_{\min}$ = 3 on Case A, and — as deterministic recomputations over the printed register rather than compiled-artifact measurements — GD(15) = 12 with $\mathrm{GD}_{\min}$ = 4 on the surgical register, whose four residual misclassifications under the best scalar rule are exactly the rows that should not be mandatory authorization gates: two runtime-safety rows, one monitoring row, and one human-process row. The comparative expert study is preregistered, hash-committed, and deferred; no comparative-superiority claim is made.**

***Index Terms*—AI governance, risk register, policy compilation, semantic refinement, runtime enforcement, runtime assurance, agentic AI, cyber-physical systems, supervised autonomy, model risk management, non-compensatory aggregation, traceability.**

From Risk Register to Runtime Predicate: A Deterministic Method for Compiling AI Governance Assessments into Enforceable Controls

ABHINANDAN GILL-LAKHOWAL[1], *Member, IEEE* (ORCID 0009-0004-2089-7262)

I. Introduction

A. The gap

Two bodies of work have matured on either side of a missing bridge.

On the governance side, NIST AI RMF 1.0, ISO/IEC 42001:2023, and financial-sector supervisory guidance (OSFI E-23, published September 11, 2025 and effective May 1, 2027; SR 11-7 in the United States) prescribe *what* organizations must do: establish accountability, profile systems, determine applicable obligations, register risks, rate and treat them, monitor them, and account for them. These frameworks are deliberately technology-neutral, and none of them prescribes a deterministic, traceable transformation from the approved risk artifacts into the predicates a runtime authority enforces. The artifact they hand to engineering is an approved risk assessment — a document.

On the enforcement side, deterministic runtime-enforcement architectures — canonically L-DREA [15] — evaluate pre-approved predicates and return PERMIT or a non-permit outcome with signed evidence. This paper's DENY (an evaluated mandatory violation) and HOLD (indeterminacy, timeout, or unresolved conflict) refine the governance provenance of that non-permit outcome — [15] distinguishes evaluated denial from SAFE_STATE under uncertainty in the same way — without modifying the downstream enforcement architecture; the phrase *safe state* is reserved here for the fallback state of an independent runtime safety layer (Section V, commitment 1). Their *initial* artifact is a predicate set — code.

In the reviewed literature we did not identify a method that specifies, with the conjunction of properties enumerated in Section II-A, how the document becomes the code. In practice the translation is performed manually, inconsistently, and without traceability: a compliance officer's risk statement ("MNPI may leak into outputs") becomes an engineer's rule ("deny if restricted-list match") through an undocumented judgment call. The mapping lives, at best, in a spreadsheet maintained by neither party; more commonly it lives nowhere. When a regulator or auditor asks *which risk this control mitigates, at what approved threshold, and on whose authority*, the answer is reconstructed after the fact — if it can be reconstructed at all.

The translation gap has three concrete failure modes, all observable in enterprise practice: (i) **silent narrowing** — an unenforceable risk statement is quietly rewritten into whatever *is* enforceable, and the residual goes untracked; (ii) **orphan controls** — rules accumulate in the enforcement layer with no surviving link to any approved requirement, so nobody can retire them and nobody can defend them; and (iii) **priority-as-authorization** — the heat-map score built for treatment sequencing is repurposed, incorrectly, as the criterion for which controls are non-negotiable. The method in this paper makes (ii) impossible *within* Φ*-produced bundles* and removes (iii) from the compiler — Φ never derives mandatory-gate membership from a scalar priority or an $L \times I$ threshold, so a score-influenced gate can exist only as an explicit, signed, attributable governance judgment, never as compiler arithmetic — with an explicit runtime acceptance condition (Section VI-F) under which the guarantee extends to a deployment — and makes (i) explicit, detectable, and measurable: the signed disposition prevents an identified remainder from being silently accepted, while whether a remainder was correctly identified is an empirical refinement-quality question (Section XII defines the silent-narrowing rate against an independently adjudicated reference).

B. Why this matters now

Four forcing functions make the translation layer urgent rather than academic:

1) **Supervisory expectations are converging on lifecycle traceability.** OSFI E-23 (published September 11, 2025; effective May 1, 2027) requires demonstrable linkage between enterprise model-risk governance and operational controls across the model lifecycle, and OSFI's July 2026 Technology Risk Bulletin applies the technology- and operational-resilience discussion to generative and agentic AI — while stating expressly that Technology Risk Bulletins are not regulatory expectations [4] — so the binding expectation is E-23's and the bulletin only shows the supervisor's attention reaching agentic deployments; the EU AI Act's staged obligations presume such linkage exists [7]. The Act's timetable was amended during the preparation of this paper: the Digital Omnibus on AI, Regulation (EU) 2026/1744, entered into force on July 27, 2026 and deferred the application of the high-risk obligations to December 2, 2027 for stand-alone Annex III systems and to August 2, 2028 for Annex I systems, while leaving the Article 50 transparency obligations — including the December 2, 2026 marking deadline — and the general-purpose-AI documentation duties applicable since August 2,

2025 unchanged [7]. The deferral moves the compliance date; it does not alter the structural expectation that an approved governance decision be demonstrably linked to the control actually in force, which is the subject of this paper.

2) **Agentic systems collapse the distance between output and effect.** When a generated artifact can invoke tools — or command an actuator — the governance-to-control gap is no longer a paperwork deficiency; it is an authorization deficiency.
3) **Runtime enforcement is now demonstrated.** With deterministic evaluation, non-compensatory aggregation, and receipt-based evidence published and reproducible [15], the binding constraint has moved upstream: the enforcement layer is only as sound as the predicates it is given.
4) **Standards bodies are soliciting exactly this bridge.** NIST's Trustworthy AI in Critical Infrastructure Profile (TACIP) Community-of-Interest discussion draft (version of Aug. 3, 2026 — an unpublished, versioned COI document, cited here by date and hash rather than as an established NIST publication [13]) names "Strategic Governance & Executive Translation" as an explicit open design area — seeking methods to map high-level governance, organizational missions, and compliance obligations onto operational practices — and its Practice 3 requires "deterministic boundary logic" whose derivation from governance artifacts it leaves unspecified. The method here is a candidate answer to that open area.

C. Contributions

Three ideas carry the paper. First, a governed refinement boundary turns human-approved risk meaning into signed control specifications, and a deterministic compiler then closes those specifications against execution authority and emits immutable, traceable runtime controls. Second, the compiler derives authorization-gate membership from consequence class and declared authority and does not read the scalar rating; an other-mandatory declaration is a signed attribute of the specification, so a reviewer who declares one because of a score has recorded a judgment, not applied a threshold. Third, the same method composes with an independent runtime-assurance/safety layer by governing *transitions* rather than *trajectories*. The first idea is the separation the rest of the paper is built on — interpretation is governed; compilation is deterministic:

$$\mathcal{G} \xrightarrow[\text{signed } J]{\Psi_K} (D, \Delta) \xrightarrow[\text{deterministic}]{\Phi} (P, G, E, B) \rightarrow \text{runtime}$$

where the left arrow is a human act over an approved catalog, recorded and signed, and the right arrow admits no judgment at all. The contributions below are the elements of those three ideas. One consequence of them is stated here rather than listed as a contribution: the method supports controlled policy evolution. A change to an obligation, a consequence classification, the derivation catalog, a threshold, or the assessed configuration is a trigger (Section III) that forces governed re-refinement under a new judgment record and deterministic recompilation into a new bundle, while the retired bundle — immutable, and retired only by a signed registry record — preserves the policy-to-control lineage under which every historical decision was taken. Policy evolution is thus executable lineage, not a research problem of its own.

- **C1 — Governed semantic refinement Ψ_K.** A refinement boundary separating human semantic judgment from mechanical compilation: an approved, versioned Control Derivation Catalog constrains how each assessed risk may be refined into signed Approved Control Specifications, and every risk receives exactly one disposition — runtime, non-runtime, accepted, or unresolved. Because equally authorized experts may lawfully complete the refinement differently, Ψ_K is formalized as a governed *relation* closed by a signed judgment record J — never as a function of the assessment alone. Determinism is claimed for what follows the boundary, never for the interpretation of unconstrained prose.
- **C2 — GC-IR with total disposition.** A machine-readable intermediate representation whose record family covers compiled predicates *and* non-runtime, accepted-risk, and unresolved dispositions, so totality is a property of dispositions (exactly one per risk), not of predicate counts — one risk may lawfully yield zero, one, or several predicates.
- **C3 — Deterministic compilation Φ.** A compiler from signed refinement outputs to a canonically serialized (RFC 8785), hashed, signed, and *immutable* predicate bundle, with authority closure against the approved action matrix under a transition-admissibility rule (discrete, latency-bounded, evidence-bounded — Section III), explicit predicate origins (risk-derived or compiler-invariant) and enforcement phases (pre-authorization, boundary re-validation, post-authorization/pre-actuation, post-event audit), mandatory-gate unknown-failure and evaluation-timeout enforcement, world-state binding of permits, an explicit precedence relation for overlapping policies, and lifecycle state (retirement, supersession, actor-scope revocation) carried in a separately signed registry rather than in the hashed payload. No best-match, similarity, or probabilistic rule selection is permitted inside Φ. GC-IR additionally represents shared, standing, delegated, and revocable authority — including N-of-M evidence requirements inside a single mandatory predicate — as schema elements (Section V, commitment 7), not as further contributions.
- **C4 — Consequence-class gate coverage, formalized over a domain-parameterized profile.** A coverage rule — membership of an approved, materiality-qualified consequence class $C^* = C^*_{\text{base}} \cup C^*_{\text{domain}}$ mandates that each authorized hazardous action path of the risk be covered by at least one mandatory gate, irrespective of likelihood $\times$ impact, while permitting supporting advisory predicates — with a gate representability theorem (exact conditions for score- and threshold-representability, and descriptor sufficiency) whose operational forms are three

propositions: monotone-threshold non-separability, score-collision non-representability, and qualified gate invariance under rating and control-effectiveness changes conditional on unchanged consequence classification. Two domain profiles are supplied: C^*_{fin} (financial services) and C^*_{cp} (cyber-physical), the latter adding reversibility-qualified physical harm to a person and accountability transfer. The enforcement-layer aggregation mathematics is cited [15], [16], not re-proven.

- **C5 — Temporally valid traceability with safety-layer composition.** An authorized-origin chain — obligation or internal requirement → risk/disposition → specification or compiler invariant → predicate → receipt — with receipt validity defined at decision time against the immutable bundle payload, the signed lifecycle registry, and actor-scope revocations; authorization-before-commit-before-actuate ordering testable as a temporal predicate; a standing-origin carve-out under which interventions by an independent runtime safety layer are logged as correlated safety events rather than as ordering violations; and a human-response record binding followed/deviated outcomes of advisory outputs to the permit that authorized them. Coverage gaps become mechanical queries.

D. Non-contributions (claim boundary, summary)

This paper does **not** claim: legal interpretation of any statute; automated determination of applicable obligations (Step 3 remains a human legal function); deterministic interpretation of unconstrained risk prose; detection capability; production deployment; trajectory or motion safety, or any substitution for a certified runtime safety layer (RTA, interlock, or safety instrumented system — the authorization gate decides whether a transition may be *attempted*, never whether the resulting motion is safe); clinical efficacy or fitness of any surgical system; comparative superiority over unaided manual practice (the comparative study is preregistered and deferred; Section XII); or that compilation eliminates the need for control validation (Step 7). Section XV states the full claim boundary in the format established in prior work.

E. Paper organization

Section II situates the method, including explicit positioning against policy-generation and machine-readable-compliance research and against runtime-assurance architectures. Section III formalizes the five-step governance input model and the transition-admissibility rule. Section IV introduces governed semantic refinement (Ψ_K, the judgment record, the catalog, specifications, and dispositions). Section V defines the GC-IR record family; Section VI the compiler Φ, bundle semantics, and the lifecycle registry; Section VII the gate coverage rule, the gate representability theorem, and its three propositions; Section VIII the temporal traceability model and the safety-layer carve-out. Sections IX, X, and XI present the three worked cases. Section XII gives the evaluation and statistical analysis plan, including the preregistered deferral of the comparative study; Sections XIII–XV the limitations, prior-work relation, and full claim boundary.

II. Background and Related Work

Governance frameworks. NIST AI RMF 1.0 (Govern/Map/Measure/Manage) [1]; ISO/IEC 42001:2023 AI management systems [2]; ISO 31000 risk vocabulary (cause–event–consequence) [3]; OSFI E-23 (model risk; published 2025-09-11, effective 2027-05-01) and B-10/B-13/E-21 plus the July 2026 generative/agentic-AI bulletin for the financial-services case [4]; SR 11-7 as the U.S. antecedent [5]. These define governance, assessment, treatment, monitoring, and accountability structure but not control derivation: none prescribes how an approved risk artifact becomes a machine-evaluable predicate, leaving that derivation to unspecified "implementation" — and none requires an artifact from which translation loss could be measured.

Policy-as-code and zero trust. XACML's subject–action–resource–condition model and OPA/Rego demonstrate machine-evaluable policy, but assume the policy already exists [8]. NIST SP 800-207's Policy Decision Point / Policy Enforcement Point split is the modern anchor [10]: GC-IR output is analogous to the policy loaded by a policy decision function, and an L-DREA-class architecture combines deterministic decision logic with an externalization enforcement point — it is not reducible to the PEP alone. GC-IR is deliberately compatible with an XACML-style target structure so compiled output can feed existing engines as well as L-DREA-class authorities. The lineage runs back to the reference-monitor concept — complete mediation, tamper-resistance, analyzability [11] — of which the compiled bundle is the *analyzable* half: the monitor's policy, produced by an auditable process rather than accreted by hand.

Process-safety antecedents. The closest field-tested analog to a compiled mandatory gate is the process-industry *permissive* (IEC 61511 / ISA-84) [12]: a precondition that must be satisfied before an action may proceed, whose trip is never traded off against process KPIs even though sensor voting (MooN) legitimately aggregates *within* a measurement channel. The method here is, in effect, a systematic way to derive AI-governance permissives from a risk register — borrowing the established discipline of permissives and interlocks rather than inventing a parallel one. The analogy also bounds itself: process permissives are derived from HAZOP/LOPA studies by trained safety engineers under a mature standard; the AI-governance equivalent of that derivation discipline is what Sections IV–VI supply.

Certification/enforcement and interlocking antecedents. The pre-action authorization gate has an engineering lineage reaching back to mid-nineteenth-century mechanical railway interlocking, through structured control tables, process-safety permissives, integrity and reference-monitor models, and contemporary machine-evaluable policy; the separation of a governed judgment stage from a mechanical enforcement stage is likewise not new. Clark and Wilson [42] divide integrity rules into *certification* — a human determination that a transformation procedure is well-formed with respect to policy, which they describe as ordinarily a manual operation — and *enforcement* by mechanism, and name reduction of the certification burden an open problem. Railway signalling has long

compiled structured control tables into executable interlocking logic, and a mature formal-methods literature generates and verifies that logic [43]. Both are antecedents of this work rather than instances of it: each begins from a specification whose executable semantics are already fixed. The question here precedes that point — whether the executable condition is a faithful and attributable derivation of the approved governance artifact from which it claims authority. Enforcement correctness and specification provenance are distinct properties, and only the latter is this paper's subject.

Runtime assurance and safety-layer composition. Cyber-physical systems already employ runtime-assurance mechanisms: Simplex-style verified fallback controllers and decision modules [35], aircraft RTA architectures [36], and, for robotically assisted surgical equipment, the safety mechanisms governed by IEC 80601-2-77 [37] within the ISO 14971 [38] / IEC 62304 [39] lifecycle. Such mechanisms may evaluate continuous safety envelopes, discrete supervisory states, mode transitions, and other conditions required to keep system behaviour within an approved safety specification. The distinction drawn here is therefore not that RTA is incapable of expressing discrete controls or authorization-like conditions. Rather, conventional RTA begins from an already specified safety property or monitor condition; it does not prescribe how enterprise obligations, risk-register judgments, authority assignments, and approved control treatments are transformed into those executable conditions with requirement-level provenance and governance lifecycle semantics. The method in this paper addresses that upstream derivation problem for consequential transitions. It determines whether an actor, capability, or system is authorized under the approved governance state to *attempt* a transition — may this capability be activated, this energy delivered, this control handed over, this subtask delegated, at all? — at event rate and only with signed authority, while an independent runtime-safety mechanism may continue to determine, at control-loop rate and with standing authority, whether the resulting behaviour remains within its safety envelope. The two functions may be implemented as separate components or composed inside an extended RTA architecture; the composition is ordered either way — a proposal passes the authorization gate before it reaches the controller, and the safety layer supervises whatever the controller then does (Section XI). The architectural claim is therefore functional rather than physical: governance-derived execution authority and runtime safety are independently justified control concerns, even where a deployment co-locates their evaluation. *Safe does not imply authorized, and authorized does not imply safe*: a permitted transition may still be stopped by the safety layer, and a safe trajectory may still be unauthorized. Ordinary consequential actuation requires both independently evaluated conditions — authorization PERMIT $\wedge$ safety-layer non-intervention — where *non-intervention* (written PASS in the outcome matrix of Section XI) denotes the safety layer's continuous supervision finding the motion admissible over the interval of the transition, not a one-shot safety predicate evaluated before actuation; the experimentally decisive cell of the outcome matrix is authorization DENY $\wedge$ safety non-intervention $\rightarrow$ no external effect, because it exhibits a class of prohibited transitions that satisfying the tested physical envelope does not identify. Section III states the admissibility rule that keeps control-loop-rate actions out of the authority matrix; Section VIII states how safety-layer interventions enter the evidence chain with standing origin rather than as ordering violations. The composition model follows the fail-closed/fail-safe composition stated in [15] and is consumed, not re-derived.

Safety cases and assurance. Goal Structuring Notation (GSN) and assurance-case practice link claims to evidence, top-down [9]. The present method is the operational complement: bottom-up refinement and compilation from register rows to enforced predicates, with receipts as the evidence GSN nodes can cite.

Model risk management. The three-lines model and SR 11-7/E-23 lifecycle stages supply the ownership and validation scaffolding assumed by Step 1 and Step 7 respectively.

Deterministic runtime enforcement. L-DREA [15] assumes an approved predicate set; this paper produces that set. Both companion papers place predicate completeness outside their own scope — "a governance-process responsibility, not a monitor-architecture responsibility" ([15], Section XII-A; [16], scope declaration) — and the present method is the governed, deterministic form of that responsibility: it supplies the mandatory predicate vector the concurrence primitive evaluates and does not alter how the primitive evaluates it. The composition with an independent safety layer follows [15], Section II-F, which assigns trajectories to Simplex and discrete actions at the actuation boundary to the authority. The interface is deliberately thin: (i) the compiled predicate bundle, which an L-DREA-class authority loads as its policy; and (ii) a requirement-reference field this paper requires conformant decision receipts to carry, so that each runtime decision joins back to its authorizing disposition — an **interface extension stated here as a conformance condition on the consuming authority**, since the permit-token and trace-record constructs of [15] do not themselves carry it (Appendix A, constraint 14). Throughout, *receipt* denotes the signed decision record (the trace record of [15]; the ERTuple of [16]) and *runtime authority* the externalization monitor of [15].

A. Positioning against policy generation and machine-readable compliance

Prior work establishes several adjacent but distinct paths from human requirements to machine-evaluable artifacts. Natural-language access-control research extracts authorization policies from requirements or user stories, but its target is normally an access-control rule and its principal validity question is extraction accuracy [18], [19]. Policy Cards express operational and regulatory constraints in a versioned machine-readable artifact suitable for runtime use [20]. OSCAL-based approaches structure AI-assurance evidence and connect policy, assessment, and enforcement records [21]. Ontological Knowledge Blocks are the closest antecedent: a deterministic regulatory compiler that translates structured intermediate records into executable SHACL constraints with provenance and evidence linkage [22]. Governance-to-deterministic-executable-control translation is therefore occupied terrain,

and this paper does not claim it. Recent pre-action authorization work specifies deterministic tool-call gating for autonomous agents at the enforcement layer [23], leaving the derivation of its policy artifacts from enterprise assessments — the question addressed here — open. The nearest upstream antecedent is Koch's layered translation method [24], which supplies a control tuple and a six-criterion runtime-enforceability rubric that decides *which layer* (governance objective, design-time constraint, runtime mediation, assurance evidence) a standards-derived objective belongs to; it begins from governance objectives, explicitly declines to compile standards into guardrails, records no signed human judgment, imposes no total disposition, and reports no evaluation. The present method does not presuppose that decision; it makes it a governed act — Ψ_K assigns every assessed risk its disposition under a signed judgment record — and then deterministically compiles the runtime remainder. Other deterministic or hash-anchored enforcement work is complementary in the same way: PCAS compiles a hand-written Datalog policy into a reference monitor with decision traces [25]; AgentSpec supplies a lightweight trigger–predicate–enforcement DSL whose rules are authored or LLM-generated [26]; P2T extracts atomic rules from prose policy with an LLM pipeline and SMT conflict detection [27]; and a deterministic control plane for coding agents anchors decisions to hashed state [28]. Each takes its policy as given or mines it from text; none derives it from a governed risk assessment under a signed judgment record with a disposition for every assessed risk, which is the object this paper produces and OAP-class authorities presuppose. These approaches demonstrate that policy authoring, compliance evidence, and executable validation can be made machine-readable; consequently, the present paper does not claim that governance-to-code translation is wholly unprecedented.

What we did not identify in any reviewed approach is a definition of translation *loss* or an instrument for measuring it; beyond that, the defensible novelty is the conjunction of nine properties, and we did not identify a reviewed approach that combines all nine, and it is against this conjunction — not against the existence of machine-readable policy — that the not-identified-in-the-reviewed-literature statement of Section I-A is made: (i) every assessed risk receives an explicit runtime, non-runtime, accepted, or unresolved disposition (risk-row total disposition); (ii) an explicit semantic-judgment boundary, so that no ordinary risk prose is silently treated as executable semantics; (iii) execution-authority action closure against an approved authority matrix and version boundary; (iv) mandatory-gate membership derived by the compiler from consequence class and declared authority, with the scalar rating excluded from that derivation; (v) DENY/HOLD execution semantics distinguishing evaluated prohibition from unresolved condition; (vi) temporally valid obligation-to-receipt lineage; (vii) ordered composition with an independent runtime safety layer; (viii) world-state-bound permits re-validated at the boundary; and (ix) delegated, standing, and concurrent authority as schema elements. Policy Cards [20] and Ontological Knowledge Blocks [22] each supply a subset of (ii)–(iii) and (vi); neither supplies (i), (iv), (v), or (vii)–(ix). Property (iv) is claimed narrowly. Gating keyed to consequence rather than to likelihood-weighted priority is not itself new: RACG gates tool exposure by discrete risk level plus a Boolean authorization variable [29], "The Controllability Trap" triggers graduated human re-authorization from a composite control-quality metric rather than from a risk ranking [30], and process-safety practice has long assigned mandatory confirmation to irreversible actions (Section II). What is claimed is mandatory *gate coverage* keyed to an approved, materiality-qualified consequence class, proven representable (Theorem 1) and embedded in a governed, signed, totally-dispositioned pipeline with requirement-to-receipt provenance. The method begins after legal applicability has been determined and ends before runtime detection or actuation performance is claimed.

III. The Five-Step Governance Input Model

We formalize the upstream assessment as five typed artifacts, collectively $\mathcal{G} = (M, S, O, R, A)$. (These correspond to Steps 1–5 of the enterprise methodology; Steps 6+ are the refinement and compilation targets and the enforcement/evidence layers of prior work.) Each artifact is serialized in a canonical machine-readable form; the refinement and compilation stages consume the serialized artifacts, never the prose documents from which they were authored.

Step 1 — Assessment metadata M = (`system_id`, `purpose`, `owner_business`, `owner_technical`, `accountable_exec`, `assessor`, `deployment`, `scope`, `exclusions`, `method`, `version_binding`, `review_cycle`, `triggers`). The critical field is `version_binding`: the exact model, prompt, dataset, and policy versions the assessment covers. Compilation output inherits this binding; a version drift event invalidates the compiled control set by construction. The canonical failure this prevents is Knight Capital (2012): a stale server-deployment flag activated retired order-router logic, producing ≈USD 440M in losses in ∼45 minutes — a non-adversarial, machine-speed version-binding failure with no malware involved [14]. A control set whose validity is cryptographically bound to `M.version_binding` renders the stale code path unauthorized by construction.

One field encodes a principle rather than a datum: `accountable_exec` designates the party who *accepts residual risk*, and that party is the accountable business authority — never the developer, the model vendor, or the AI-governance team that performed the assessment. The compiler enforces the separation mechanically: escalation routes and threshold approvals resolve to M ownership fields, so a bundle in which the assessor and the acceptor are the same identity fails structural validation.

`M.triggers` enumerates the events that force reassessment: model version change, prompt or policy change, dataset refresh outside declared bounds, scope expansion, new obligation, catalog change (Section IV-B), consequence-class reclassification (Section VII-F), or incident above a declared severity. Because every compiled record carries `version_binding_ref`, a trigger firing does not merely *recommend* re-review — it retires the bundle through the lifecycle registry (Section VI-F), with temporal validity semantics defined in Section VIII.

Step 2 — System profile S = (`capabilities`, `data_classes`, `actors`, `affected_parties`, `authority_matrix`, `architecture`, `scale`, `autonomy_level`). The decisive distinction is **capability vs. authority**: what the system *can* generate versus what it is *authorized to cause*. The authority matrix — actor $\times$ action $\times$ resource $\times$ destination — is the vocabulary from which predicate subjects, actions, resources, and destinations are drawn. If an action does not appear in `S.authority_matrix`, no predicate can permit it; the fail-closed default of the enforcement layer handles the remainder. The matrix therefore does double duty: it is both a governance artifact (the accountable executive's signature over what the system may cause) and a compiler symbol table (the closed vocabulary Φ resolves against).

Transition admissibility. The authority matrix admits *transitions*, not trajectories. A row (actor, action, resource, destination) is admissible only if it satisfies:

- **AD-1 (discrete transition, not feedback primitive).** Two conditions, both required. *Semantic condition*: the action is proposed as an event with a declared pre-state and post-state, and it changes a discrete authority, mode, capability, resource, or externally effective state whose meaning is independent of the controller's periodic feedback update. *Timing condition*: the action is not a primitive of a feedback loop whose safe operation depends on meeting that loop's deadline — equivalently, the authorization latency of the action does not become part of a certified or safety-relevant feedback path. Formally, $\mathrm{AD1} = \mathrm{DiscreteStateTransition} \wedge \neg\mathrm{FeedbackControlPrimitive}$. Proposal rate is diagnostic evidence, not the definition: because a system carries several loops with different periods, `S.architecture` declares `control_loops` = {(`control_loop_ref`, `control_loop_period`, `excluded_action_family`)}, one entry per excluded action family, and a proposal class whose inter-proposal interval is at or below the `control_loop_period` of the loop its action family belongs to — force scaling, haptic feedback, servo tracking, envelope-keeping corrections — is presumptively a feedback primitive and is excluded under the reference `control_loop_ref`. A slow corrective loop is still continuous regulation, and a fast discrete mode command is still a transition; the semantic condition decides, and the period declaration records why.
- **AD-2 (latency-bounded).** Every Approved Control Specification over the row declares an `evaluation_latency_bound` (with threshold contract, Section IV-C) within which all of its conditions can be evaluated; expiry of the bound is HOLD (`on_evaluation_timeout`), never a partial permit (compiler invariant INV-LATENCY, Section VI-B).
- **AD-3 (evidence-bounded).** The row has a corresponding actuation record class, so that authorization $\rightarrow$ commit $\rightarrow$ actuation (Section VIII) is testable for it.

A register row whose only enforceable observable would require an AD-1-violating action receives reason code **RC-06 — continuous control; belongs to the runtime safety layer** — citing the `control_loop_ref` under which it was excluded — and a non-runtime disposition routed to the `runtime_safety` control family (Section V, commitment 5). This is not a gap in coverage; it is the method declining to place a servo loop behind a permission gate, which would make the gate the slowest element in a control path it is not certified to be in. `S.architecture` correspondingly declares `safety_layers`: for each independent safety mechanism (RTA, interlock, SIS) that supervises the system's physical behaviour, its identifier, standards basis, standing-authorization reference, and `safety_event_window` Δ_{se} — the declared bound, under a threshold contract, within which a signed `SafetyEvent` must be recorded after an intervention (Section VIII; Appendix A, constraint 21). `S.autonomy_level` may additionally record a capability ladder (data $\rightarrow$ insight $\rightarrow$ guidance $\rightarrow$ augmentation $\rightarrow$ supervised autonomy [44]; Case C, Appendix D), so that the control set scales with the tier actually deployed. `DELEGATE` and `TRANSFER_CONTROL` are catalog transitions available to any profile whose authority matrix includes a human or automated actor that may hand authority to another (Section V, commitment 7).

Two profile-time judgments condition everything downstream. First, `S.autonomy_level` answers the scoping question *is this truly an agent?* — a system that only summarizes documents is a generative application; one that plans, selects tools, retrieves, and performs actions across systems carries materially higher agentic risk, and the profile records which it is so that the control set scales with the answer rather than with the marketing label. Second, where human review appears in the authority matrix it must be *substantive*: an approver clicking through without time, evidence, or authority is not oversight. GC-IR reflects this in two places — the `human_attestation` basis tests only the presence and validity of the signed attestation artifact (Section V), and the substantive-ness of the review behind that artifact is monitored by a separate, deterministic proxy (reviewer dwell/volume telemetry, Case A row R-12) rather than assumed.

Step 3 — Obligation matrix O = {(`obligation_id`, `source`, `jurisdiction`, `territorial_basis`, `clause_ref`, `requirement_text_hash`, `applicability_basis`, `applicability_decision`, `decision_authority`, `decision_time`, `effective_from`, `effective_until`, `legal_source_version`, `requirement_class`)}. Applicability is determined by legal entity, jurisdiction, users, data, intended use, output destination, and performable actions — a human legal determination whose record is now first-class rather than merely asserted. Each row carries the clause-level anchor (`clause_ref`), a hash of the requirement text as applied (`requirement_text_hash`), the applicability decision itself with its deciding authority and decision time, the validity interval (`effective_from`, `effective_until`), and the version of the legal source

consulted (`legal_source_version`) — so that a later amendment to the source is detectable as a hash or version mismatch rather than a silent divergence. The method consumes O as ground truth and never interprets legislation; the expanded record makes the ground truth auditable and temporally valid. Each obligation carries a `requirement_class` $\in$ {`statutory`, `supervisory`, `standards`, `contractual`, `internal`} used later in gate selection; `standards` denotes a consensus standard (ISO, IEC, ASTM) applied by the profile, which the Step 3 applicability decision may promote to `statutory` where a regulator incorporates it by reference.

Step 4 — Risk register $R = \{r_i\}$, each r_i = (`cause`, `event`, `consequence`, `affected_parties`, `obligation_refs` $\subseteq O$, `existing_controls`, `owner`). The cause–event–consequence triple is mandatory: it is what makes a register row *refinable*. "Hallucination risk" is not refinable; "because the model generates probabilistic text, unsupported financial facts may be presented as true, causing defective research" is — the cause names the mechanism to be constrained, the event names the phenomenon whose observable must be selected, the consequence names the class that drives gate selection. This is ISO 31000's own vocabulary [3] applied with compiler-grade strictness: the register author is not asked to learn a new language, only to finish sentences in the existing one.

Step 5 — Risk analysis $A = \{(r_i, L_i^{\text{inh}}, I_i^{\text{inh}}, e_i, L_i^{\text{res}}, I_i^{\text{res}}, \texttt{consequence\_class}_i, \texttt{tier}_i, \texttt{treatment}_i)\}$ on the enterprise taxonomy, where the analysis separates three concepts: **inherent risk** $(L^{\text{inh}}, I^{\text{inh}})$ — exposure before controls; **control effectiveness** e_i — whether existing controls are appropriately designed *and* operating; and **residual risk** $(L^{\text{res}}, I^{\text{res}})$ — exposure remaining after effective controls. Tiering, treatment, and review priority are driven by residual ratings, as enterprise practice requires. Where the treatment is acceptance, the analysis row carries the acceptance authority (resolving to `M.accountable_exec`), scope, and expiry that the accepted(RA) disposition of Section IV-D consumes; each row may additionally declare a per-risk reopening trigger ρ_i — an event class that forces re-analysis of that row without a full reassessment.

Note that `consequence_class` is recorded *separately* from any impact rating, and it is the element of A that ordinary control effectiveness does not move: controls reduce *likelihood* (and sometimes realized impact); they do not, by lowering a score, change *what kind of consequence* the event is. An unauthorized trade behind three effective controls is still an unauthorized trade. Section VII-F states this as a *qualified* invariance: consequence class can change only through formal, governed reclassification (for example, when a control alters the action pathway itself), never as an automatic consequence of a lower residual score. Where an enterprise taxonomy records only L and I, adopting the method requires exactly one schema addition — the `consequence_class` column — and one governance act — executive approval of the C^* profile (Section VII).

IV. Governed Semantic Refinement

A. Why a refinement boundary is required — and why refinement is a relation, not a function

An ordinary cause–event–consequence statement is not itself a predicate. It does not necessarily identify the runtime attribute, evidence producer, action tuple, operator, threshold, temporal window, or failure response needed for deterministic evaluation. Treating free-form text as though it contained those semantics would merely hide a human judgment inside a function named `extract_observable`.

The method therefore separates governed semantic refinement from deterministic compilation. Let $\mathcal{G} = (M, S, O, R, A)$ denote the approved governance assessment. Because two equally authorized experts, constrained by the same catalog, may lawfully produce different approved specifications from the same assessment, refinement is **not a function of** $\mathcal{G}$: writing $\Psi_K : \mathcal{G} \to (D, \Delta)$ would smuggle a determinism claim over human interpretation that the method explicitly disavows. Refinement is a **governed relation**, closed into a single-valued mapping only by the signed judgment record J — the set of signed selections (assigned event types, chosen templates, completed control fields, approvals) that the governance authorities actually made:

$(D, \Delta) \in \Psi_K(\mathcal{G})$, with $\Psi_K(\mathcal{G}, J) \to (D, \Delta)$ single-valued given J. Figure 1 shows the boundary this places between judgment and determinism. Refinement is bounded by disposition completeness, not predicate completeness: failure to obtain an approved executable representation for a risk does not authorize Ψ_K to invent one, and the remaining semantics must be routed to an approved non-runtime control, formally accepted, or recorded as unresolved (Section IV-D). No assessed risk may disappear between assessment and compilation.

J is itself a governance artifact: versioned, signed, included in the authorized-origin chain, and referenced structurally — every record of D and Δ carries `judgment_ref` = (`J.id`, `J.version`, `J.hash`), and Φ verifies that reference and includes `J.hash` in the canonical payload, so that the lineage from assessment through judgment to compiled predicate is checkable rather than asserted. This construction preserves determinism for everything downstream of the boundary — Φ is a function of signed inputs whose refinement products are fixed by J — without implying deterministic human interpretation of prose. Ψ_K is not a legal interpreter and does not infer requirements from unconstrained natural language. Human governance authorities assign an approved `event_type`, select an allowed template from the Control Derivation Catalog K, complete the required control fields, and sign the resulting specification; those signed acts *are* J. The catalog constrains what may be selected; the judgment record establishes who authorized the selection. An LLM may assist upstream of J — proposing a candidate structured control from prose — but its output is a candidate, never a signed act, and nothing in Φ or in the runtime authority depends on it.

The deterministic compiler is then

$$\Phi : (M, S, O, R, A, K, C^*, J, D, \Delta, \text{INV}) \to (P, G, E, B)$$

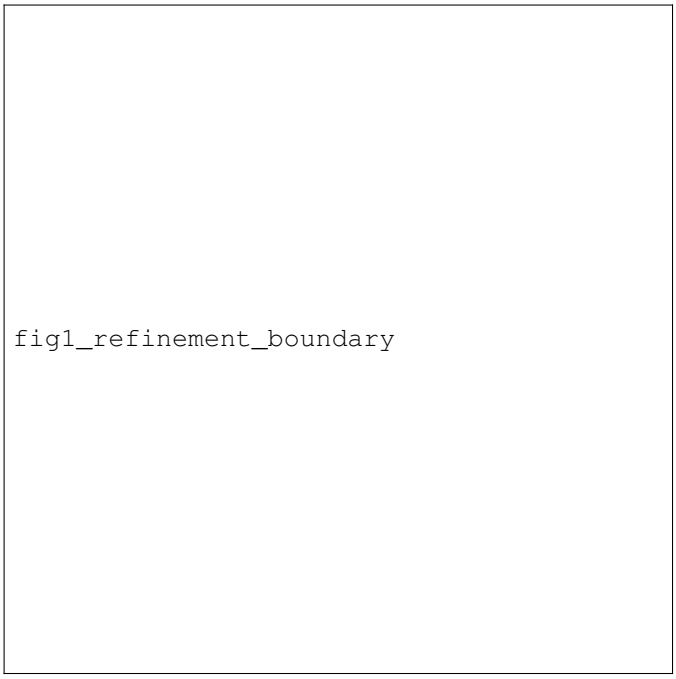

Fig. 1. The refinement boundary. Semantic judgment is exercised once, under an approved catalog K and recorded in the signed judgment record J; everything to the right of (D, Δ) is deterministic and admits no judgment.

where J enters only as the signed record whose hash the products D and Δ cite, P is the predicate set, G the gate map, E the escalation map, and B the canonical signed bundle. The determinism claim applies to Φ, not to unaided human interpretation of prose.

B. Control Derivation Catalog

Each catalog entry $k_j \in K$ is: k_j = (`event_type`, `observable_class`, `allowed_producers`, `triple_template`, `allowed_operators`, `value_schema`, `evaluation_basis`, `evidence_schema`, `permitted_responses`).

The catalog is approved by the governance forum, versioned with M, canonically serialized, and included in the compiled-bundle hash. Addition, removal, or semantic alteration of a catalog entry is a reassessment trigger. A register row whose approved `event_type` does not resolve to a catalog entry is not interpreted heuristically; it receives an unresolved or non-runtime disposition.

C. Approved Control Specification

For risk r_i, each approved runtime control $d_{ij} \in D_i$ contains: d_{ij} = (`acs_id`, `risk_id`, `obligation_refs`, `event_type`, `observable_id`, `observable_producer`, `evidence_source`, `subject`, `action`, `resource`, `destination`, `parameter_schema`, `operator`, `expected_value`, `temporal_window`, `evaluation_basis`, `threshold_contract_ref`, `enforcement_phase`, `evaluation_latency_bound`, `on_evaluation_timeout`, `on_unknown`, `on_fail`, `concurrence_expiry_response`, `gate_candidate`, `mandatory_role`, `state_binding`, `concurrence_policy`, `standing_permit_ref`, `delegation`, `input_provenance`, `escalation`, `evidence_requirements`, `control_owner`, `approver`, `approval_time`, `validity_interval`).

`enforcement_phase` $\in$ {`PRE_AUTHORIZATION`, `BOUNDARY_REVALIDATION`, `POST_AUTH_PRE_ACTUATION`, `POST_EVENT_AUDIT`} declares *when* the runtime authority evaluates the condition. Only `PRE_AUTHORIZATION` and `BOUNDARY_REVALIDATION` conditions contribute to PERMIT; a `POST_AUTH_PRE_ACTUATION` condition (receipt commit) is evaluated at interlock release and blocks release on failure; a `POST_EVENT_AUDIT` condition (safety-event correlation) is evaluated after the event and, on failure, places HOLD on the next ordinary authorization in the affected scope — it never gates the event it audits. The field removes an ambiguity that would otherwise let a post-authorization obligation masquerade as a pre-authorization predicate. Three failure responses are kept distinct: `on_fail` (an evaluated condition violated $\rightarrow$ DENY for mandatory gates), `on_evaluation_timeout` (the evaluator itself did not finish inside `evaluation_latency_bound` $\rightarrow$ HOLD), and `concurrence_expiry_response` (a time-complete requirement — the concurrence window — expired unsatisfied $\rightarrow$ DENY). Evaluator lateness and requirement expiry are different events and never share a field.

`observable_producer` identifies the component or person that creates the evidence. This field prevents a deterministic predicate from being confused with the possibly probabilistic process that generated its input. For example, a runtime predicate may deterministically verify that a signed entity-resolution artifact reports no restricted-list match; it does not thereby claim that the upstream entity resolver is perfectly accurate. `mandatory_role` $\in$ {`decisive`, `supporting`} records, for C^*-classified risks, whether this specification is a decisive blocking gate or a supporting advisory/weighted condition (Section VII).

`state_binding` declares the tuple of world-state attributes — for example (`procedure_step`, `instrument_id`, `port_configuration`, `patient_position_hash`), or, in a transaction domain, (`transaction_id`, `amount`, `beneficiary`, `account_status`, `token_status`) — over which a permit issued under this specification is digested. Version binding (Section III) ties a permit to the *configuration* the assessment covered; state binding ties it to the *situation* the authorizer saw. A permit is valid at the execution boundary only if the digest of the declared attributes recomputed there equals the digest at authorization time; a mismatch is HOLD and forces re-authorization (INV-STATE-BINDING). Case B's single-use permit bound to the transaction hash is the degenerate instance in which the state tuple is the payload itself; Case C generalizes it.

`concurrence_policy` = (`n_required`, `m_eligible`, `role_constraints`, `all_or_nothing`

= true, window) generalizes single-signer attestation, with concurrence_expiry_response = DENY. Partial concurrence never accrues: either n distinct attestations satisfying the role constraints are present within the window, or the window expires and the outcome is DENY — an expiry, not an evaluator timeout, which would be HOLD. standing_permit_ref names a class-scoped StandingPermit (Section V, commitment 7) that may satisfy the human_attestation basis for the matched transition types; it never satisfies state, safety, or context conditions. delegation = (parent_permit_id, scope $\subseteq$ parent.scope, validity $\subseteq$ parent.validity, actions $\subseteq$ parent.actions, delegatee) is present only on specifications over the DELEGATE transition; strictness is defined on the composite: (scope, validity, actions)$_{\text{child}} \subsetneq$ (scope, validity, actions)$_{\text{parent}}$, i.e., containment on every component and inequality on at least one, so an identical child is not a delegation. input_provenance enumerates, for a consequential decision, the permit identifiers of the advisory outputs on which it rests (Section VII-I).

Each Approved Control Specification carries three express statements:

1) **Evidence semantics** — what the input evidence represents and who produced it.
2) **Decision semantics** — the exact condition evaluated by the runtime authority, including the enforcement phase, which failure produces DENY (an evaluated prohibition, on_fail; an expired concurrence window, concurrence_expiry_response) and which produces HOLD (a required condition unresolved, on_unknown; an unfinished evaluation, on_evaluation_timeout).
3) **Warrant boundary** — what a passing result establishes and what it does not establish.

D. Total risk disposition

Total here means that every assessed risk receives exactly one authorized terminal disposition and therefore remains accounted for even when it yields no executable predicate. Totality is a property of governance disposition, not of predicate coverage: a risk may yield zero, one, or several predicates without violating the invariant, so $|R| = |P|$ is neither implied nor asserted. Three completeness properties are kept distinct throughout — total disposition (did any assessed risk disappear?), origin closure (did any predicate appear without authorized provenance? every predicate resolves to an approved risk or compiler invariant, Eq. (1)), and C^* coverage (did any mandatory hazardous action path escape gating? Section VII) — and are measured by DC, OPR, and CV respectively (Section XII). Every risk receives exactly one disposition: $\Delta_i \in \{$ runtime(D_i), nonruntime(C_i), accepted(RA_i), unresolved(U_i) $\}$.

- **runtime**(D_i) binds the risk to one or more Approved Control Specifications.
- **nonruntime**(C_i) binds it to approved architectural, contractual, human-process, or runtime-safety controls.
- **accepted**(RA_i) records formal residual-risk acceptance, scope, expiry, and acceptor.
- **unresolved**(U_i) records a reason code and blocks release of the affected authority scope.

A bundle is release-admissible only if $\forall r_i \in R,\ \Delta_i \neq$ unresolved, and every acceptance remains within its approved validity interval. Totality is: $\forall r_i \in R,\ |\Delta(r_i)| = 1$. Predicate cardinality is deliberately independent of risk cardinality: one risk may produce zero, one, or several predicates without invalidating totality.

E. Refinement fidelity

The compiler warrants that emitted predicates faithfully encode the signed Approved Control Specifications. It does not warrant that those specifications completely represent the real-world hazard. Semantic fidelity between R and D is separately evaluated against an independent adjudicated reference in Section XII.

Silent narrowing is defined as follows. Let Q_i^* be the set of control-relevant semantic elements required by the adjudicated reference for risk r_i, and let Q_i be the elements preserved in its approved disposition. Then:

$$SN_i = \mathbf{1}[\, Q_i^* \setminus Q_i \neq \emptyset \ \wedge\ \neg\mathrm{Covered}_i \,],$$

where $\mathrm{Covered}_i$ holds iff an explicit residual or non-runtime disposition covers the difference. The definition has a property worth stating plainly: narrowing is not the failure; *silence* is. A risk lawfully narrowed to an enforceable core, with the residual explicitly routed to a non-runtime control or formally accepted, scores $SN_i = 0$; only the untracked difference counts. Nor is the narrowing malicious — it is the rational act of an engineer who must ship something evaluable, performed in a pipeline that has no place to write down what was lost. Total disposition is that place: under it the dropped semantics must land somewhere — in the runtime specification, in an explicit non-runtime routing, in a signed acceptance — or their absence is a discrepancy the reference standard detects. The method does not claim to make $SN_i = 0$ by syntax alone; it makes narrowing detectable and measurable.

V. GC-IR: The Governance-to-Control Intermediate Representation

GC-IR is a record *family*, not a single record type: Record = CompiledPredicate $\oplus$ NonRuntimeDisposition $\oplus$ AcceptedRiskDisposition $\oplus$ UnresolvedDisposition (oneOf).

A non-runtime, accepted, or unresolved row is a first-class record and is not required to contain subject, action, resource, evaluation basis, or gate type. A compiled predicate record carries, among the fields normatively listed in Appendix A, the requirement reference (risk and obligation identifiers), the action tuple, the context conditions each with its evidence producer and on_unknown response, the evaluation basis, the enforcement phase, the evaluation-latency bound and timeout response, the gate type and source, the failure response, the escalation route, the evidence requirements, and the version and

validity bindings; a complete example record is reproduced in Appendix A. Design commitments:

1) **Every context condition declares `on_unknown`, and every specification declares `on_evaluation_timeout` — indeterminacy and evaluator lateness are named failure classes, distinct from violation and from requirement expiry.** Missing, stale, or schema-invalid context is *indeterminacy*, distinct from violation, and operationally more frequent than clean violation (sensor dropout, stale retrieval, schema drift, an unavailable screening service). A control that only trips on explicit out-of-range values silently passes it. The outcome semantics are therefore: **PERMIT** — every applicable pre-authorization mandatory condition was resolved and satisfied; **DENY** — at least one applicable mandatory prohibition, or time-complete requirement such as a concurrence window, was definitively violated or expired unsatisfied; **HOLD** — authorization cannot presently be established because required evidence or state is missing, stale, schema-invalid, contradictory, unavailable, or changed (a state-binding mismatch), or because the evaluation itself did not finish within its latency bound, or because applicable mandatory policies conflict without an approved precedence. Both non-permit outcomes are fail-closed — no externalization occurs — and both are non-permit outcomes of the consuming authority [15] (DENY maps to its evaluated denial; HOLD to its SAFE_STATE, controlled non-execution under uncertainty); the distinction lives in the receipt, where an auditor must be able to tell an evaluated denial from an unevaluated one, and in the escalation route, where a HOLD enters a verification workflow that a DENY does not. The phrase *safe state* is not used for either: it denotes the fallback state of the independent safety layer (Section II), which this method never commands. For mandatory gates `on_unknown = HOLD` and `on_evaluation_timeout = HOLD` are required, not default (Section VI-A), and `concurrence_expiry_response = DENY` is fixed; for non-mandatory conditions any fail-open exception must be explicitly approved and carries the approver's identity. This propagates the enforcement layer's fail-closed semantics — and its fail-closed (authorization default) vs. fail-safe (process outcome) distinction [15] — upstream into the governance record itself.
2) `evaluation_basis` **is constrained to** {`deterministic_lookup`, `threshold_on_measured_value`, `structural_check`, `human_attestation`}. A GC-IR condition may deterministically evaluate a signed numeric output produced by a probabilistic model — a fraud score, a confidence value — under an approved threshold contract; the determinism claim applies to the comparison and the authorization rule, not to the epistemic correctness or repeatability of the evidence producer. Deterministic runtime evaluation does not make the upstream estimator deterministic: *probabilistic evidence ≠ probabilistic authorization*. A fraud score of 0.002 or a model confidence of 99.9% is evidence a policy may consult through a threshold contract; it is never authority. `human_attestation` is deterministic at evaluation time: the predicate tests for the *presence and validity of a signed attestation artifact*, not for the quality of the human judgment it records. The `evidence_producer` field (from the ACS) keeps the deterministic check and the possibly probabilistic evidence source explicitly separate.
3) `gate_type` ∈ {`mandatory`, `weighted`, `advisory`} **with recorded** `gate_source`. Only mandatory conditions participate directly in non-compensatory aggregation. Section VII governs assignment; `gate_source` ∈ {`C_STAR`, `OTHER_MANDATORY`, `SOFT`} keeps C^*-selected gates distinguishable from mandatory gates created for other approved reasons. A record marked `weighted` is structurally invalid unless it carries `aggregation_group`, a positive weight w_i, a normalized deficit function v_i, a group threshold, and a response. At the enforcement boundary, a weighted group resolves to a **single group-level deficit** — the group's threshold test — which then enters the non-compensatory aggregate as one dimension like any other; compensation is admissible only *inside* an approved group, never across the mandatory aggregate, consistent with the aggregator-class separation proved in [15], [16]. Customer tenure, account balance, device history, and model confidence may be excellent; a single prohibited beneficiary is still DENY.
4) **Every threshold ships with a threshold contract.** A condition may be evaluated over a bounded, declared history — a rolling-window count or exposure budget, as Case B row B-03 does — and the window is then part of the contract; unbounded history is not admissible, because it would make the predicate's evidence unreproducible at replay. Accordingly, authorization need not be memoryless: individually admissible transitions may become inadmissible when an approved bounded-history predicate over their cumulative count, exposure, resource consumption, or state evolution is violated. A numeric or temporal bound is not admissible until it carries: operational definition, numerator and denominator, evidence source, ground truth, threshold rationale, uncertainty method, unit, and reproduction procedure. (Rate-style quantities with different denominators are never used interchangeably; zero-event results are reported with the exposure denominator and an exact one-sided upper bound, never as "zero risk.") Threshold *authority* is separated from evaluation: control owners propose thresholds with rationale, the accountable governance forum approves them, and the runtime evaluator never sets or adjusts them — the governance half of a two-clock separation that prevents the implementation from grading itself.

5) **Non-runtime disposition is first-class.** A risk that does not yield a runtime predicate receives a NonRuntimeDisposition (or AcceptedRisk / Unresolved) record with a typed reason: `RC-01 no per-action observable`, `RC-02 no authority-matrix action`, `RC-03 requires probabilistic judgment`, `RC-05 consequence class undefined`, `RC-06 continuous control --- belongs to the runtime safety layer` (fails AD-1, Section III); an unresolved obligation reference raises warning code `WC-01` (an unresolved reference is not simultaneously a successful compilation and a rejection, so it is a warning, not a disposition). The `routed_to` control family of a NonRuntimeDisposition is one of {`architectural`, `contractual`, `human_process`, `runtime_safety`}; a `runtime_safety` routing names the safety layer in `S.architecture.safety_layers` and the standing-authorization invariant INV-SAFETY-RECOVERY under which that layer acts. Rows are never silently narrowed into something enforceable-but-different. A non-runtime disposition is not a defect of the method — it is the method correctly reporting that a requirement belongs to a different control family than runtime authorization. Reason and warning codes are closed at schema v1.1; RC-06 and the record classes of commitments 7–8 are the v1.0 → v1.1 extension, made as the schema-version event that v1.0 required.
6) **The hashed payload is immutable — lifecycle state lives outside it.** No field of a compiled record or bundle payload may change after signing. Retirement, supersession, and revocation are represented as separately signed records in the lifecycle registry (Section VI-F), never as mutation of a `retired_at` or any other payload field; a payload that begins with a null lifecycle field and is later edited would change the bundle hash and break every receipt that cites it.
7) **Authority may be shared, pre-issued, and narrowed — never widened.** Three record forms extend single-signer attestation. A `concurrence_policy` (Section IV-C) is all-or-nothing N-of-M with role constraints and deny-on-window-expiry. It is not the predicate-level concurrence of [16] — under which every mandatory predicate must concur and a quorum threshold is a compensatory operator to be rejected — but a within-predicate evidence rule: n distinct attestations are the evidence condition of one mandatory predicate, and that predicate enters $\Gamma = \max_i d_i$ as a single dimension. Nothing here aggregates across predicates. A `StandingPermit` = (`permit_class_id`, `transition_types`, `issuer`, `roles`, `validity_interval`, `concurrence_ref`?, `signature`) is matched by transition *type* against the authority matrix, never by situation, and satisfies only the `human_attestation` basis of its matched types; every other condition of every specification over those types — state binding, safety, context, version — is still evaluated per transition. Its validity interval may not exceed the assessment validity interval. Whether a standing permit may cover a C^*-classified transition at all, and under what issuance policy, is a *profile* decision rather than a GC-IR theorem: both supplied profiles require such a permit to be issued under a concurrence policy, on the reasoning that pre-issuing attestation for a class of consequential transitions is itself a consequential authority act, and a profile that chose otherwise would record and justify that choice in the same governed field. `DELEGATE` issues a child permit whose composite (`scope`, `validity`, `actions`) is strictly contained ($\subsetneq$) in the parent's and which cites `parent_permit_id`; a child equal to or not contained in its parent is rejected at issuance (INV-DELEGATION-NARROWING), and revocation of a parent revokes its children transitively.
8) **Advisory outputs carry a human-response record.** Where a specification's `gate_type` is `advisory`, or where the transition is the presentation of guidance rather than actuation (Case C, `PRESENT_GUIDANCE`), the conformant runtime emits a `HumanResponse` = (`permit_id`, `response` ∈ {`followed`, `deviated`, `no_response`}, `responder`, `response_time`, `signature`) against the permit that authorized the output. The record denies nothing; it is evidence. It is what makes Section VII-I operable: a consequential decision's `input_provenance` lists advisory permit identifiers, and the human-response records on those identifiers show whether the operational picture was accepted or overridden before the decision was taken.

Full normative schema requirements in Appendix A.

VI. Deterministic Compilation and Bundle Semantics

A. Compiler definition

For approved inputs, Φ performs no natural-language inference (Appendix B gives the reference semantics). The unit of compilation is the Approved Control Specification: each ACS yields one or more `CompiledPredicate` records as determined by its catalog template (`instantiate_predicates` in Appendix B), and records are not merged across specifications, so the number of risk-derived records in a bundle is at least the number of approved specifications compiled into it. It validates references, resolves catalog templates, instantiates predicates, applies the gate coverage rule, records dispositions, and emits a canonical payload. For each d_{ij}, Φ shall:

1) Verify the signatures and version bindings of M, S, O, R, A, K, J, D, and Δ, and that every record of D and Δ cites `J.hash`.
2) Resolve `event_type` to exactly one approved catalog template.
3) Resolve (subject, action, resource, destination) against `S.authority_matrix`.

4) Validate action parameters against `parameter_schema`.
5) Validate evidence type and producer against the selected catalog entry.
6) Require a threshold contract for every numeric or temporal condition.
7) Assign the gate source and gate type under Section VII.
8) Require `on_unknown = HOLD` and `on_evaluation_timeout = HOLD` for mandatory gates, `on_fail = DENY`, and `concurrence_expiry_response = DENY` wherever a concurrence policy is present.
9) Require a declared `enforcement_phase`; admit only `PRE_AUTHORIZATION` and `BOUNDARY_REVALIDATION` conditions into the PERMIT aggregate, and bind `POST_AUTH_PRE_ACTUATION` and `POST_EVENT_AUDIT` conditions to interlock release and to the next-authorization HOLD respectively.
10) Bind escalation, evidence, and validity requirements.
11) Emit the predicate and its authorized origin.
12) Verify transition admissibility (AD-1..AD-3) for every action tuple against `S.architecture.control_loops`; reject any specification over an inadmissible tuple (it must instead carry RC-06 with its `control_loop_ref`).

At bundle level, Φ additionally verifies the C^* **coverage requirement** (Section VII) — every risk with $C^*(c_i) = 1$ must have at least one mandatory gate, or mandatory gate set, covering each of its authorized hazardous action paths — and rejects any bundle in which coverage fails. No best-match, similarity, or probabilistic rule selection is permitted inside Φ.

B. Predicate origins and compiler invariants

Every emitted predicate carries `origin_type` $\in$ {`risk_derived`, `compiler_invariant`}. A risk-derived predicate cites its `risk_id` and Approved Control Specification. A compiler-invariant predicate cites an approved system requirement such as INV-VERSION (per-action version-binding check), INV-EVIDENCE-COMMIT (receipt committed after authorization and before actuation), INV-AUTHORITY-CLOSURE (action tuple within the approved matrix), INV-STATE-BINDING (state digest at the execution boundary equals the digest at authorization, else HOLD), INV-LATENCY (an independent deadline watchdog, outside the evaluator it supervises, asserts HOLD with no partial permit if it expires before the evaluator signals completion within `evaluation_latency_bound`), INV-DELEGATION-NARROWING (every child permit strictly contained in its parent), or INV-SAFETY-RECOVERY (interventions and recovery motions of a declared safety layer carry standing authorization, bypass the authorization gate, and are logged as `SafetyEvent` records correlated to the authorization state in force). Each invariant carries its enforcement phase: INV-VERSION, INV-AUTHORITY-CLOSURE, INV-LATENCY, and INV-STATE-BINDING are `PRE_AUTHORIZATION` with `BOUNDARY_REVALIDATION`; INV-DELEGATION-NARROWING is evaluated at issuance (`PRE_AUTHORIZATION` of the child); INV-EVIDENCE-COMMIT is `POST_AUTH_PRE_ACTUATION`; INV-SAFETY-RECOVERY is `POST_EVENT_AUDIT`. Two further distinctions matter here. INV-LATENCY and INV-EVIDENCE-COMMIT are *meta-enforcement* invariants ($P_{\text{meta}} \subset P$) rather than ordinary authorization predicates (P_{auth}): an evaluator that fails to complete cannot evaluate a predicate stating that it failed, so INV-LATENCY is discharged by a deadline watchdog that sits outside the predicate evaluation it supervises (the watchdog conjunct of the consuming authority's interlock, [15], Section V-F), and INV-EVIDENCE-COMMIT is discharged by the commit-before-actuate interlock, not by the evaluator; Φ emits both as invariant records so that they carry ownership, a threshold contract, escalation, and evidence requirements, and the runtime acceptance condition (Section VI-F) requires the consuming authority to realize them by mechanisms independent of the evaluator. The last two are not pre-authorization predicates and cannot contribute to PERMIT; they are represented as predicates so that their failure is an evaluated, receipted outcome — blocked release, or HOLD on the next ordinary authorization — rather than an unrecorded process defect. Formally, $P = P_{\text{auth}} \sqcup P_{\text{meta}} \sqcup P_{\text{standing}}$: P_{auth} holds the `PRE_AUTHORIZATION` and `BOUNDARY_REVALIDATION` records that enter the non-compensatory PERMIT aggregate; $P_{\text{meta}} = \{\text{INV-LATENCY}, \text{INV-EVIDENCE-COMMIT}\}$ holds records carrying `meta_enforcement` = true, discharged by mechanisms independent of the evaluator and excluded from that aggregate; $P_{\text{standing}} = \{\text{INV-SAFETY-RECOVERY}\}$ holds the standing-origin record audited post-event. PERMIT is a function of P_{auth} and the independent deadline watchdog alone; eligibility to actuate additionally requires the committed receipt that INV-EVIDENCE-COMMIT names. Origin closure holds over all of P:

$$\forall p \in P, \qquad \text{origin}(p) \in R \cup \text{INV}. \tag{1}$$

The partition is a labelling of one record set, not three output vectors; Appendix B emits every member into P with its phase and marker. Compiler invariants are not orphan controls; they are separately authorized requirements included in M or S and recorded in the approved invariant register INV. The traceability requirement is:

$$\forall p \in P,\ \text{origin}(p) \in R \cup \text{INV}.$$

C. Canonicalization and determinism

Canonical payload serialization follows a declared canonicalization standard — RFC 8785 JSON Canonicalization Scheme [31] — with UTF-8 encoding; canonical object-key ordering; deterministic array ordering where order is not semantically meaningful; explicit numeric representations; explicit units; no local time, locale, or floating-point-environment dependencies; no timestamp or random nonce inside the hashed payload; and **no mutable lifecycle field inside the hashed payload** (Section VI-F).

The semantics are designed to exclude environment-dependent sources of nondeterminism; whether an implementation achieves that across independently installed hosts is a measurement (Section XII-F), not a consequence of the design. Let $C(x)$ be canonical serialization and H be SHA-256. Translation determinism for supported execution environments e, e' and any semantics-preserving input permutation π is:

$$TD(x) = \mathbf{1}[\, H(C(\Phi_e(x))) = H(C(\Phi_{e'}(\pi(x))))\,].$$

TD applies to the canonical payload hash. The signature envelope may include signing time, key identifier, and certificate material and therefore is not required to be byte-identical across signing events.

D. Policy conflict and precedence

Overlapping predicates are resolved through an explicit precedence relation:

mandatory DENY $\succ$ mandatory HOLD $\succ$ mandatory pass $\succ$ weighted/advisory result.

No weighted or advisory result may override a mandatory DENY or HOLD. Conflicts among applicable mandatory policies — one control requiring an action that another prohibits — are themselves indeterminate and produce HOLD, with both control identifiers, versions, and the conflict state recorded in the receipt, unless a unique precedence relation is present in the approved policy metadata; the authority never averages the two or invents an undocumented precedence. The safety layer sits outside this relation entirely: its interventions are not predicate results and are not aggregated (Section VIII).

E. Totality and origin-closure assertions

The implementation shall assert $|\{\Delta_i : r_i \in R\}| = |R|$ and $\forall r_i \in R,\ |\Delta(r_i)| = 1$. It shall **not** assert $|P| + |X| = |R|$, because one risk may lawfully produce multiple predicates (and compiler invariants produce predicates with no register parent at all — lawfully, via INV). The corresponding origin-closure assertion is Eq. (1), taken over all of P including its meta-enforcement and standing-origin members; the pseudocode's `all_predicates_have_authorized_origin` check (Appendix B) is its implementation.

F. Bundle lifecycle, immutability, and the runtime acceptance condition

The bundle payload is **immutable after signing**. Retirement, supersession, emergency revocation, and actor-scope revocation are represented by separately signed records in a **lifecycle registry** REG:

`reg_entry` = BundleLifecycle(`bundle_hash`, `action` $\in$ {retire, supersede, revoke}, `effective_time`, `authority`, `successor_hash`?, `signature`) $\oplus$ ActorScopeRevocation(`actor_id`, `scope` $\subseteq$ `S.authority_matrix`, `effective_time`, `authority`, `cascade_to_children`, `signature`)

— never by mutating the payload. Bundle-level records retire policy content; actor-scope records withdraw a named actor's authority over a named scope from an effective time ("the surgeon withdraws authority," "the analyst's publication right is suspended," "the customer freezes the account") without touching the bundle, and void every standing or delegated permit that actor holds in that scope, transitively. Validity at a time t is therefore a joint check of the payload hash, the bundle registry, and the actor-scope registry (Section VIII): a bundle whose payload hash verifies but which REG retires before t is invalid at t, and a bundle mutated in place fails the hash check outright. This matches the immutability discipline of the consuming enforcement layer, whose hash-chained evidence records likewise never mutate committed state [15]; a trigger event of `M.triggers` retires the bundle by appending a signed registry record, not by editing the bundle. The registry also separates the two change clocks that a physical deployment must keep apart: enterprise policy — a new contraindication, a changed privilege, a revised consequence class — is recompiled and re-released through the registry, while the certified safety-layer implementation, which never consulted the bundle, need not itself be modified; whether the resulting system-level change requires reassessment or recertification remains deployment- and assurance-case-dependent.

The orphan-control guarantee of Section I-A is, precisely stated, a **compiler guarantee**: within Φ-produced bundles, every predicate has an authorized origin, verified structurally. It extends to a *deployment* only under an explicit **runtime acceptance condition**: the runtime authority loads only bundles that verify as signed Φ outputs — signature, payload hash, and lifecycle-registry state — and accepts no policy content through any other path. Where an operator can inject controls through a separate policy channel, the guarantee remains scoped to the compiled bundle and the deployment does not inherit it. The acceptance condition is therefore stated as a deployment conformance requirement (Appendix A, constraint 14), not assumed.

VII. Consequence-Class Gate Coverage

The problem. Enterprise practice rates risks by likelihood $\times$ impact and prioritizes by the product. But authorization is not prioritization. A rare unauthorized trade may score $L = 2$, $I = 5 \rightarrow 10$, landing mid-matrix; its consequence is nonetheless non-negotiable.

A. Operational consequence descriptor and the coverage rule

A consequence descriptor is $c_i = (\mathrm{kind}_i, \mathrm{materiality}_i, \mathrm{reversibility}_i, \mathrm{authority}_i)$, and the approved mandatory-class predicate is $C^*(c_i) \in \{0, 1\}$. C^* is a governance object, versioned with M, and is **domain-parameterized**: $C^* = C^*_{\mathrm{base}} \cup C^*_{\mathrm{domain}}$, where

C^*_{base} = { `material_statutory_prohibition`, `unauthorized_authority_exercise`, `protected_data_disclosure_outside_consent_scope`, `irreversible_external_effect_above_approved_bound` },

and each deployment approves exactly one domain extension. C^* is an execution-authority consequence profile over AD-1-admissible discrete transitions: a hazard whose only actionable pathway is excluded by AD-1 as a feedback-control primitive does not become an authorization gate merely because its physical consequence is severe — it receives RC-06 and is governed by the declared runtime-safety layer (Section XI, C-06 and C-07). The two profiles used in the worked cases are:

- C^*_{fin} (Cases A and B): $(C^*_{\text{base}} \setminus \{\texttt{protected\_data\_disclosure\_outside\_consent\_scope}\}) \cup \{\texttt{material\_information\_barrier\_breach}\}$ — the finance data-plane instance replaces the generic disclosure element rather than extending it, so that the profile remains a set and the replacement is itself an approved governance act.
- C^*_{cp} (Case C): $C^*_{\text{base}} \cup \{$ `physical_harm_to_person`(reversibility $\in$ {irreversible, requires_intervention}), `accountability_transfer` $\}$.

`physical_harm_to_person` is reversibility-qualified: harm that is self-limiting and requires no intervention does not enter C^* on its class alone, exactly as an immaterial documentation defect does not enter on its statutory source alone — materiality qualifies membership. `accountability_transfer` covers any transition that changes who is answerable for the system's next consequential act — control handoff, delegation to autonomy, mode change — because the harm is not physical but evidential: an act whose responsible party cannot be named afterward. Stating C^* as a base plus a domain extension is what answers Limitation 6: generality is no longer argued only from the neutrality of GC-IR and K but demonstrated by a second profile over a physically actuating system.

Each classification records the approving authority, rationale, materiality boundary, effective date, and any authorized emergency-override procedure. The C^*-derived gate is $g_i^C = C^*(c_i)$; the final approved gate may also reflect other mandatory policies: $g_i = g_i^C \vee g_i^{\text{other}}$. The compiler records `gate_source` so C^*-selected gates remain distinguishable from mandatory gates created for other approved reasons. Both inputs to the approved gate are declared attributes of the row, fixed when the classification is approved: c_i — including materiality_i, which is a categorical determination against the approved materiality boundary rather than the impact rating I_i — and g_i^{other}, which is set by an approved policy that does not condition on (L_i, I_i, s_i) or on the control-effectiveness value e_i; a policy that gated by score would be a threshold rule and is excluded from g_i^{other} by construction. Neither varies under re-rating; either changes only through re-approval of the classification. More generally, the compilation discipline is domain-neutral while its governance inputs are domain-parameterized: C^*_{domain}, obligation sources, authority matrices, evidence producers, threshold contracts, control-loop and safety-layer declarations, concurrence policies, and escalation rules may all vary by sector or deployment, and a change in any of them reaches the runtime only as a new governed input, a new signed refinement, a recompilation, and a new immutable bundle version — never as semantic improvisation inside Φ. Enterprises may extend C^* (the profile is a governance object, versioned with M), but contraction within an active assessment is itself a trigger event requiring re-approval.

> **Coverage rule (CV).** For any risk r_i with $C^*(c_i) = 1$, the approved specification set D_i must contain at least one mandatory gate — or mandatory gate set — covering **each authorized hazardous action path** associated with r_i, **irrespective of** $L \times I$. A C^* risk may, in addition, generate supporting advisory or weighted predicates: C^* membership mandates *coverage*, not mandatory status for every predicate derived from r_i (the ACS `mandatory_role` field records which specifications are decisive and which are supporting). Φ verifies CV structurally and rejects non-covering bundles. Heat-map score may continue to govern review priority, monitoring depth, and treatment sequencing, but it is not an input to C^*-derived coverage, and Φ never derives mandatory-gate membership from the scalar score; any score-influenced OTHER_MANDATORY status is therefore an explicit, signed governance judgment, not compiler arithmetic.

In the theorem and propositions below, $g_i = 1$ denotes that risk r_i carries mandatory-gate coverage under the approved assignment.

B. Comparison with a declared heat-map rule

Let $s_i = L_i I_i$ and let the enterprise's predeclared heat-map gate rule be $h_{T_H}(i) = \mathbf{1}[s_i \geq T_H]$. Divergence against that declared rule is $GD(T_H) = \sum_i \mathbf{1}[g_i \neq h_{T_H}(i)]$. To determine whether *any* scalar threshold can reproduce the approved gate map, define:

$$GD_{\min} = \min_{t \in \mathcal{T}} \sum_i \mathbf{1}[g_i \neq \mathbf{1}[s_i \geq t]],$$

where $\mathcal{T}$ contains the distinct observed scores and boundary values immediately above and below them.

C. The gate representability theorem

The three propositions that follow are the operational forms of one characterization. Say the approved assignment g is *score-representable* if there exists $h : \mathbb{R} \to \{0,1\}$ with $g_i = h(s_i)$ for all i, and *threshold-representable* if there exists t with $g_i = \mathbf{1}[s_i \geq t]$ for all i. Call a pair (i, j) a *collision* if $s_i = s_j$ and $g_i \neq g_j$, and a *weak inversion* if $s_i \leq s_j$, $g_i = 1$, and $g_j = 0$ (a *strict inversion* if $s_i < s_j$).

Theorem 1 (Gate representability). (a) g is score-representable if and only if R contains no collision. (b) g is threshold-representable if and only if R contains no weak inversion. (c) Threshold-representability implies score-representability; the converse fails whenever R contains a strict inversion but no collision. (d) The class-based assignment $g_i = C^*(c_i) \vee g_i^{\text{other}}$ is well-defined on every register, regardless of collision or inversion structure, and is invariant under all

changes to likelihood, impact, and control-effectiveness values that leave each c_i's approved classification and each g_i^{other}'s declared status unchanged (both are declared attributes, Section VII-A).

Proof. (a) Necessity: a function is single-valued, so $s_i = s_j$ forces $h(s_i) = h(s_j)$, which a collision contradicts. Sufficiency: absent collisions, $h(s) := g_i$ for any i with $s_i = s$ is well-defined on the observed scores, and $h := 0$ elsewhere completes it. (b) Necessity: if (i, j) is a weak inversion and t exists, $g_i = 1$ requires $t \le s_i$, whence $s_j \ge s_i \ge t$ forces $\mathbf{1}[s_j \ge t] = 1$, contradicting $g_j = 0$. Sufficiency: absent weak inversions, every gated score strictly exceeds every ungated score; if no row is gated any $t > \max_i s_i$ represents g, otherwise $t := \min\{s_i : g_i = 1\}$ gates every gated row by construction and, since every ungated score lies strictly below t, gates no ungated row. (c) A threshold rule is a function of s, giving the implication; a register with a strict inversion and no collision is score-representable by the pointwise h of (a) but not threshold-representable by (b). (d) $C^*(c_i)$ reads no likelihood, impact, or effectiveness value, so it is constant under their variation; g_i^{other} is fixed by hypothesis — a declared attribute of the approved row that is not a function of L_i, I_i, or e_i (Section VII-A) — so both operands of the disjunction are unchanged and g_i is invariant. Well-definedness follows because C^* is a total predicate on descriptors and g_i^{other} is declared for every row. ■

Corollary (sufficiency of the descriptor). Two cases must be kept apart. If R contains a weak inversion, no monotone threshold on the scalar score reproduces the assignment (Proposition 1, Theorem 1(b)); a non-monotone function of the score may still do so (Theorem 1(c)), but such a function ranks a lower-scoring risk above a higher-scoring one and is no longer a priority rule. If R contains a collision with divergent membership, no function of the scalar score alone — monotone or otherwise — reproduces the assignment (Proposition 2, Theorem 1(a)). In the first case any correct rule that remains monotone in priority, and in the second case any correct rule at all, must incorporate additional information sufficient to distinguish the offending rows; the approved consequence descriptor supplies such information, since the class-based assignment of (d) is well-defined on every register. The corollary claims sufficiency, not minimality: a smaller discriminator may separate a particular offending pair, and nothing in Theorem 1 shows the four-component descriptor to be the least such input. $GD_{\min} \ge 1$ whenever a weak inversion is present.

Propositions 1 and 2 are the necessity halves of (b) and (a); Proposition 3 is (d). The mathematics is elementary by design. Its value is *regulatory decidability*: Theorem 1 turns "should this control be non-negotiable?" from an argument about scores into a checkable structural property of the approved register, and $GD(T_H)$ turns disagreement with heat-map practice into a number.

D. Monotone threshold non-separability

Proposition 1 (Monotone threshold non-separability). If there exist risks r_i, r_j such that $s_i < s_j$, $g_i = 1$, and $g_j = 0$, then no monotone threshold rule $h_t(s) = \mathbf{1}[s \ge t]$ reproduces both assignments, and therefore $GD_{\min} \ge 1$.

Proof. If $t \le s_i$, then $s_j > s_i \ge t$, so $h_t(s_j) = 1$, contradicting $g_j = 0$. If $t > s_i$, then $h_t(s_i) = 0$, contradicting $g_i = 1$. Thus every t misclassifies at least one of the pair. ■

Case A contains such an inversion: R-10 is gated at score 10 while R-04 and R-12 are not gated at score 12. Case C contains one at the opposite end of the map: C-01 is gated at score 8 while C-06 — a continuous-control row routed to the safety layer — is not gated at score 16. Case B demonstrates consequence-based selection but, because all listed rows are mandatory, is not used by itself to prove $GD_{\min} > 0$ (Section X).

E. Score-collision non-representability

Proposition 2 (Score-collision non-representability). If there exist risks r_i, r_j with $s_i = s_j$ and $g_i \ne g_j$, then no rule $h(s)$ that is a function of the scalar score alone — monotone or otherwise — reproduces the approved gate assignment.

Proof. Any function of s satisfies $h(s_i) = h(s_j)$ whenever $s_i = s_j$; the approved assignment has $g_i \ne g_j$, so h disagrees with it on at least one of the pair. ■

Proposition 1 excludes monotone thresholds; Proposition 2 is stronger on its narrower premise: under a score collision with divergent gates, class-dependent gating is not representable by *any* function of the score, because the information that separates the two risks — the consequence descriptor — is not present in the score at all. Stated as an information-loss result: *a scalar risk score erases consequence information that the approved authorization decision requires*. This is the paper's central formal result about heat maps. It is a statement about representational sufficiency, not about policy correctness: the propositions establish that the score is informationally insufficient for the approved gate assignment, not that C^* is universally the right policy — the substantive correctness of a consequence-class assignment remains a governance judgment (Limitation 5). Case A instantiates the collision directly: R-09 ($s = 12$) is gated by class membership while R-04 and R-12 ($s = 12$) are not. Case C instantiates it four ways at $s = 12$ (Section XI).

F. Qualified gate-invariance proposition

Proposition 3. Conditional on an unchanged action model, consequence descriptor, C^* definition, and approved mandatory-policy set, gate membership is invariant under changes to likelihood, impact ratings, or ordinary control-effectiveness estimates: $g_i(L, I, e) = g_i$ for all admissible (L, I, e) that do not trigger approved consequence reclassification — informally, $\partial g/\partial L = \partial g/\partial I = \partial g/\partial e = 0$ under frozen classification.

This does not claim that consequence class can never change. A control that changes the action *pathway* — for example, replacing irreversible actuation with a reversible queue — may justify formal reclassification. Such a reclassification is a governance decision and version-trigger event, not an automatic consequence of a lower residual-risk score. The contrast with a heat-map rule is the point: under any $L \times I$

threshold, improving controls lowers residual L, the product falls below the threshold, and the gate silently disappears exactly when confidence in the control environment is highest — gates decaying as a function of their own past success. Under the C^* rule, the gate survives every re-rating and disappears only through an explicit, attributable reclassification decision — a signed governance act, not an arithmetic side effect. Authorization gates should be removed by people taking responsibility, never by multiplication.

G. Rating sensitivity

Because enterprise likelihood and impact ratings are ordinal, the robustness of heat-map gate membership to rating uncertainty is assessed by discrete Monte Carlo sampling (Section XII-E). A continuous mean-value bound on the surrogate $s = L \cdot I$, which explains why scalar membership is unstable near a threshold, is recorded in Appendix E as explanatory only.

H. Normalized non-compensatory evaluation

Each mandatory condition has a nonnegative violation function $d_i = v_i(m_i, \theta_i) \geq 0$, where $v_i = 0$ means satisfaction and $v_i > 0$ means violation; this general form supports upper bounds, lower bounds, equalities, set relations, and Boolean conditions. The mandatory aggregate is $\Gamma = \max_i d_i$, with PERMIT $\Leftrightarrow \Gamma = 0$. A weighted aggregate with positive tolerance $\tau > 0$ can admit a sufficiently small individual violation; this statement does not apply to every conceivable weighted rule — a positive-weight sum authorized only when its value is exactly zero is also non-compensatory for nonnegative deficits. The claim is therefore limited to compensatory weighted schemes with positive authorization tolerance, and compensation is admissible only inside an approved aggregation group whose boundary result enters Γ as a single deficit (Section V, commitment 3). The compiled mandatory predicate vector is consumed by the non-compensatory authorization semantics established in [16] and instantiated at the externalization boundary by [15]; their aggregation proofs, ablations, and runtime-enforcement measurements are prior work and are not reproduced here. This paper's contribution is strictly upstream: determining, through governed refinement and deterministic compilation, which approved conditions populate that vector.

I. Cumulative decision influence (boundary case)

The coverage rule attaches to a risk whose authorized action path crosses the execution boundary. A distinct case arises where no single artifact crosses it and the aggregate does: individually reversible outputs assemble into the picture on which a consequential decision is taken. The disposition rule is: aggregation is in scope only where the consuming decision is already designated consequential under the approved impact tiering; the control attaches to that decision as a declared `input_provenance` field enumerating the AI-generated contributions on which it rests, each with its evidence producer; and the residual — a declared set complete as declared but incomplete in fact — is an evidence-production exposure

fig2_origin_chain

Fig. 2. The authorized-origin chain and its temporal validity condition. A risk disposed as non-runtime or accepted remains traceable without producing a predicate; receipt validity is a joint check of signature, payload hash, bundle registry, actor-scope registry, and state digest at decision time.

(RC-03) routed to a non-runtime disposition and stated in the warrant boundary. This is a boundary case, not a further contribution; it asserts no proposition, is exercised by Case C rows C-03, C-08, and C-13; whether it warrants a consequence class of its own is the open question recorded in Section XIII.

VIII. Temporal Traceability and Evidence Binding

The authorized-origin chain, with its temporal validity condition, is shown in Figure 2:

(obligation or internal requirement) → risk/disposition → ACS or compiler invariant → predicate → receipt.

An obligation that cannot be represented by a runtime predicate remains traceable through its approved non-runtime or accepted-risk disposition. Traceability completeness does not require every obligation to produce a predicate.

For receipt t, bundle b, and lifecycle registry REG, define:

$$\begin{aligned}\mathrm{ValidAt}(t, b, \mathrm{REG}) = {} & \mathrm{SignatureValid}(t) \\ & \wedge\ t.\mathit{hash} = H(b.\mathit{payload}) \ \wedge\ b.\mathit{from} \leq t.\tau \\ & \wedge\ \neg\mathrm{Retired}(\mathrm{REG}, H(b.\mathit{payload}), t.\tau) \\ & \wedge\ \neg\mathrm{RevokedScope}(\mathrm{REG}, t.\mathit{actor}, t.\mathit{action\_tuple}, t.\tau) \\ & \wedge\ \mathrm{StateValid}(t, b),\end{aligned}$$

where $t.\tau$ is the receipt's decision time, $t.\mathit{hash}$ its bundle hash, $b.\mathit{from}$ the bundle's `effective_from`, and $t.\mathit{digest}$ the state digests it records; $\mathrm{Retired}(\mathrm{REG}, h, \tau)$ holds iff REG contains a validly signed retirement, supersession, or revocation record for h with `effective_time` $\leq \tau$, and $\mathrm{RevokedScope}(\mathrm{REG}, a, x, \tau)$ holds iff REG contains

a validly signed ActorScopeRevocation for actor a whose scope contains action tuple x with `effective_time` $\leq \tau$; and $\mathrm{StateValid}(t, b)$ holds iff no state binding is declared under b for the predicates t evaluates, or $t.digest_{\mathrm{boundary}} = t.digest_{\mathrm{auth}}$ — digest equality is required exactly where a binding is declared (INV-STATE-BINDING), not universally. Because the payload is immutable (Section VI-F), the hash check binds the receipt to exactly the policy content in force; the registry checks supply the lifecycle and authority state without touching that content; and the state-digest check binds the receipt to the situation the authorizer saw. A historical receipt is not drift merely because its bundle is now retired or its actor's authority has since been withdrawn. Drift occurs when the bundle was not valid, the actor's scope was already revoked, or the bound state had already changed at the decision time, or when actuation used a bundle after its registry-effective retirement. The time-of-check/time-of-use exposure — a proposal evaluated under valid authority that reaches the externalization boundary after the token is revoked, the beneficiary is changed, the account is frozen, or the control owner changes — is closed by re-evaluation at the boundary, not by trusting the proposal-time check: the state digest and the registry are consulted where the effect occurs.

Where commit-before-actuate is claimed, the temporal predicate is:

$$t_{\mathrm{authorization}} \leq t_{\mathrm{evidence\ commit}} < t_{\mathrm{actuation}}$$

— the authorization decision is taken, the receipt recording that decision is committed to the evidence chain, and only then may actuation occur. This matches the enforcement pipeline of [15], in which permit issuance precedes the trace-record commit that precedes interlock release; a receipt cannot ordinarily be committed before the authorization decision that creates it. Actuation is domain-parameterized: instruction released to a payment rail, authorization response transmitted, ledger-changing transaction committed, order transmitted to a venue, instrument activated, energy delivered.

Safety-layer carve-out. The ordering predicate is stated over `AuthorizedActuation` records — actuations that followed a permit. A physical system with a declared safety layer also produces motions that followed no permit: an interlock stop, a retraction to a safe pose, a fallback-controller takeover, an emergency stop. These are not ordering violations; they are the safety layer exercising the standing authority declared in INV-SAFETY-RECOVERY, and the architecture never requires an emergency mechanism to request permission and wait before reverting to its certified safe state. They are recorded as

SafetyEvent = (`safety_event_id`, `safety_layer_id` $\in$ `S.architecture.safety_layers`, `trigger`, `effective_time`, `origin` = INV-SAFETY-RECOVERY, `correlated_permit_id`?, `correlated_bundle_hash`, `signature`)

— with standing origin rather than a per-event permit, and correlated to the authorization state in force when they fired (the permit, if any, under which the interrupted transition was proceeding). Two consequences follow. First, the transition the safety event interrupted does not resume on its old permit: the state digest has changed, so INV-STATE-BINDING returns HOLD and re-authorization is required — the authorization layer learns of the intervention through the evidence chain, not through a command channel. Second, an audit query that fired on every safety stop would be measuring the wrong thing; the query is therefore scoped, and a separate query tests the safety events themselves. The evidence obligation on the safety layer is temporal and post-event: $\mathrm{SafetyIntervention} \Rightarrow \Diamond_{\leq \Delta_{\mathrm{se}}} \mathrm{SignedSafetyEvent}$ for a declared recording window Δ_{se}, and $\mathrm{MissingSafetyCorrelation} \Rightarrow \mathrm{HOLD}$ on the next ordinary authorization in the affected scope — the gate never conditions the intervention itself.

The ten audit queries that operationalize these conditions — from obligations without a disposition through advisory permits lacking a human-response record — are listed in Appendix A (audit queries). Empty result sets demonstrate linkage and temporal-integrity completeness under the declared data model. They do not, by themselves, establish legal compliance, semantic correctness, control effectiveness, or the safety of any trajectory. The permit-binding and single-use semantics that make an unbound, replayed, or substituted authorization detectable are specified in [15] and consumed here unchanged; what this paper adds is the requirement-reference, disposition, human-response, and safety-correlation content that conformant receipts must carry (Section II; Appendix A, constraints 14–15 and 21).

IX. Case Study A — Investment Research Agent (Financial Services)

(Condensed here; the full Step 1–5 artifact set, catalog, specifications, judgment record, and dispositions ship machine-readable in the repository. Case is synthetic but fully specified: every field required by Sections III–IV is populated, and the artifact is hash-pinned so refinement and compilation are reproducible by reviewers.)

Profile highlights. RAG summarizer over an approved research library; ∼220 analysts, ∼3,200 briefs/month; draft-only authority (analyst approval required); no trading connectivity in scope; exclusions explicit. Obligations (Step 3, Canadian dealer): OSFI B-13/E-21/B-10/E-23-readiness; CIRO Rule 3600 research supervision, with individual predicates citing the applicable sections (notably ss. 3608–3622) rather than the rule number alone [6]; privacy statutes; securities/information-barrier requirements. Consequence profile: C^*_{fin}.

Table I gives the register. Rows R-01–R-14 correspond one-to-one to the fourteen-row source register; R-15 and R-16 are register extensions added to exercise the remaining reason-code classes. Dispositions: 13 runtime, 3 non-runtime, 0 accepted, 0 unresolved — DC = 1 by construction once dispositions are signed; RCY = 13/16 (descriptive). Every C^* row satisfies the coverage rule CV: each carries at least one decisive mandatory gate over its authorized hazardous action path, and supporting advisory conditions (where present) are recorded with `mandatory_role = supporting`. WC-

01 and RC-05 handling are exercised in the machine-readable artifact's deliberately seeded validation rows.

R-01 note. The predicate is deterministic only because the output schema represents factual claims as *typed claim objects* — structured fields carrying a claim identifier, a text span, and a source-hash slot — so "every factual claim resolves to a source" is a structural check over enumerated objects, never a semantic classification of prose. Identifying what constitutes a factual claim in *unstructured* text would require probabilistic judgment and is exactly the RC-03 condition; that residual — untyped prose read as a factual assertion despite falling outside the claim schema — is routed upstream to the evidence-production layer and to R-12 monitoring, and the warrant boundary in the ACS states it. Where an adopting system cannot type its claims, R-01 lawfully receives a non-runtime disposition rather than a pretend-deterministic predicate.

R-09 note. "Unauthorized recommendation" receives a runtime disposition *deterministically* because the predicate never judges whether prose "sounds like" advice — that would be RC-03. Its catalog template tests structure: recommendation-class fields (rating, price target, action-language slots in the output schema) are absent from the authority matrix and therefore structurally unpermittable, and the mandatory reliance disclaimer is a structural presence check. The residual risk that free prose is *read as* a recommendation despite the disclaimer is a human-factors exposure routed to R-12 monitoring — an honest warrant boundary, stated in the ACS rather than papered over.

R-11 note. The model/vendor-change row is where a register requirement and a compiler invariant meet: Φ emits INV-VERSION for every bundle by construction, so R-11's predicate coincides with a control that exists regardless. The register row is not redundant — it attaches the *governance* apparatus (owner, rating, treatment, escalation) to the invariant, and it converts the Knight Capital argument of Section III from a bundle-invalidation policy into a per-action runtime test: no action is authorized under a configuration the assessment never covered. Origin metadata keeps the two authorizations distinct (`risk_derived` citing R-11; `compiler_invariant` citing INV-VERSION).

Reading of Propositions 1 and 2 on this register (Figure 3)**.** R-10 ($s = 10$) and R-13 (10) sit at the bottom of the runtime rows' heat map — below R-01 (20) and below R-02/R-03/R-11 (16) — yet carry mandatory gates by class. Conversely R-04 (12) and R-12 (12) outrank R-10 on the map yet correctly receive no gate, while R-09, at the same score of 12, is gated by class membership alone. The inversion ($s_{\text{R-10}} = 10 < 12 = s_{\text{R-04}}$, $g_{\text{R-10}} = 1$, $g_{\text{R-04}} = 0$) satisfies Proposition 1, so $GD_{\min} \geq 1$ on this register; and the three-way collision at $s = 12$ (R-04, R-09, R-12 with gates 0, 1, 0) instantiates Proposition 2 — no function of the scalar score, monotone or otherwise, reproduces this assignment. Measured on the pinned artifact, against the enterprise's predeclared threshold $T_H = 15$: $GD(15) = 3$ — R-09, R-10 and R-13 carry mandatory gates by class while scoring below the threshold — and $GD_{\min} = 3$, so no scalar threshold reproduces the approved assignment more closely than the declared one. Both sums run over all sixteen register rows, non-runtime dispositions included: R-14, R-15, and R-16 carry residual scores of 12, 6, and 8 (stated with their dispositions below) but no gate, and excluding them would understate divergence at low thresholds — with them, $GD(t) = 3$ at $t \in \{13, 14, 15\}$ and at $t \in \{9, 10\}$, $GD(t) = 4$ at $t \in \{7, 8\}$, and $GD(t) \geq 5$ elsewhere, so every figure above is recomputable from the printed rows alone. The R-14, R-15, and R-16 residual ratings are governed fixture values carried in the released judgment record rather than measured quantities; over the range of plausible alternative ratings for those three rows, $GD_{\min}$ would range from 2 to 5, while $GD(15) = 3$ is unaffected so long as the three remain below T_H. The compiled Case A bundle (payload SHA-256 `f5cbc3a8016159d2074005fa021bf76ca04fb7aed1408f5ec845418460a43536`) carries 19 predicate records — 16 risk-derived from the thirteen runtime rows and 3 compiler invariants — against 12 obligation rows, with OPR $= 0/19$ and PTC $= 19/19$.

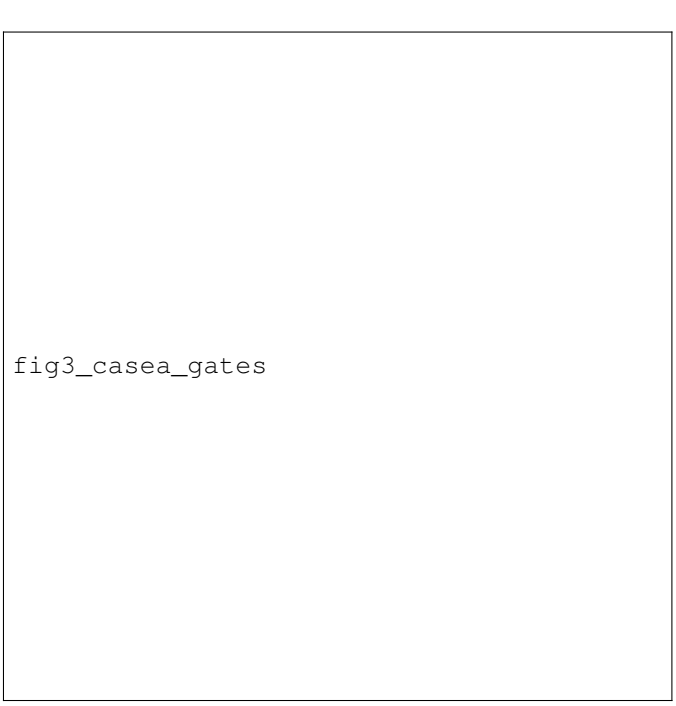

Fig. 3. Case A gate assignment against residual rating. Three C^*-classified risks are gated while scoring below the enterprise's declared heat-map threshold, giving $GD(15) = 3$; the inversion at (10, 12) instantiates Proposition 1 and the collision at $s = 12$ instantiates Proposition 2.

Non-runtime dispositions (three rows, two reason codes).

- **R-14 — concentration/resilience risk** (single-vendor dependence of the retrieval stack; residual $L \times I$ = 12 (L3·I4); no gate). **RC-01: no per-action observable** — no event evaluable at authorization time distinguishes a concentrated architecture from a diversified one. Disposition: nonruntime (architectural/contractual — multi-vendor clause, resilience testing, OSFI B-10 alignment). Honest non-runtime disposition, not forced translation.
- **R-15 — unprofessional or reputationally damaging tone** (register extension; residual $L \times I$ = 6 (L3·I2); no gate). **RC-03: requires probabilistic judgment** — "tone" has no approved catalog observable and no ap-

TABLE I
CASE A: REGISTER → DISPOSITION → PREDICATE (EXCERPT). THIRTEEN ROWS RECEIVE RUNTIME DISPOSITIONS; THREE RECEIVE NON-RUNTIME DISPOSITIONS (TEXT). $L \times I$ VALUES ARE RESIDUAL RATINGS. GATE SOURCE C^* DENOTES C^*_{fin}.

risk_id	Event (phenomenon → approved observable)	consequence_class	$L \times I$	Disposition / gate (source)	Predicate sketch	on_unknown
R-01 hallucinated facts	typed claim object lacks resolving source hash	client_harm	20	runtime · mandatory (other)	every typed claim object in the output schema resolves to an approved-library source hash (structural check)	HOLD
R-02 citation failure	cited source $\notin$ approved library	client_harm	16	runtime · mandatory (other)	citation set $\subseteq$ approved library manifest	HOLD
R-03 stale data	source timestamp $>$ freshness bound	client_harm	16	runtime · mandatory (other)	max source age $\leq$ policy bound per asset class	HOLD
R-04 biased framing	coverage skew metric $>$ bound	conduct	12	runtime · weighted + advisory	measured skew $\leq$ threshold (contract; aggregation_group)	warn
R-05 MNPI leakage	restricted-list match in context/output	C^*: info-barrier (material)	15	runtime · **mandatory** (C^*)	signed entity-resolution artifact reports restricted_list_match == false	HOLD + escalate
R-06 missing conflict disclosures	mandatory disclosure absent	C^*: statutory (material)	15	runtime · **mandatory** (C^*)	disclosure checklist complete	HOLD
R-07 prompt injection / poisoning	retrieved doc fails provenance/structure check	security	15	runtime · mandatory (other)	source provenance verified; tool-call $\notin$ retrieved content	HOLD
R-08 unauthorized publication	distribution without supervisor approval token	C^*: authority	15	runtime · **mandatory** (C^*)	valid supervisory approval bound to this draft hash	HOLD
R-09 unauthorized recommendation	recommendation-class fields present, or reliance disclaimer absent	C^*: authority	12	runtime · **mandatory** (C^*)	recommendation-class fields $\notin$ authority matrix → structurally unpermittable; reliance disclaimer present (structural check)	HOLD
R-10 unauthorized trading	order-system call attempted	C^*: irreversible	10	runtime · **mandatory** (C^*)	action $\notin$ authority matrix → structurally unpermittable	HOLD
R-11 model/vendor change	runtime version fingerprint $\neq$ assessed configuration	governance (bundle validity)	16	runtime · mandatory (other; coincides with INV-VERSION)	runtime model/prompt/policy fingerprint == M.version_binding	HOLD
R-12 weak oversight	reviewer dwell/volume outside band	governance	12	runtime · advisory → monitoring	review-time telemetry within band (deterministic proxy for substantive review)	warn
R-13 evidence failure	receipt chain gap	C^*: statutory (material)	10	runtime · **mandatory** (C^***; coincides with INV-EVIDENCE-COMMIT**)	receipt committed after authorization and chained before release	HOLD

proved threshold contract: a tone classifier's score could in principle be evaluated deterministically under commitment 2 of Section V, but no such operationalization (observable, producer, threshold, rationale, uncertainty method) has been proposed to or approved by the governance forum, so the row lawfully remains RC-03. The obstacle is the absence of an approved operationalization, not a categorical bar on probabilistic evidence. Disposition: nonruntime (human-process review workflow). A *derived* deterministic proxy (measured style-guide violation count under a threshold contract) may be proposed to the governance forum as a new catalog entry and register row — a governance act, not a compiler shortcut.

- **R-16 — third-party model vendor commercial viability** (register extension; residual $L \times I = 8$ (L2·I4); no gate; distinct from R-11's *technical* vendor-change risk, which compiles). **RC-01** — vendor solvency has no authorization-time observable. Disposition: nonruntime (contractual — escrow, exit clause). The R-11/R-16 pair is instructive: the same vendor generates one runtime and one non-runtime risk, and the method separates them where manual practice conflates them.

X. CASE STUDY B — TRANSACTION AUTHORIZATION AGENT (BANKING)

(Purpose: a second, structurally different domain — high-volume, machine-speed, per-event authorization — and the case for which enforcement-layer evidence already exists. This revision makes the upstream chain explicit for the predicate family that [15] evaluated at volume, so that the primary handoff object between the two papers is the executable predicate bundle, with the requirement-reference receipt field as its evidence-side counterpart.)

Profile highlights. Per-transaction authorization agent; machine-speed decisioning; every proposed externalization (payment release) mediated by the runtime authority; authority matrix admits exactly one external action (`release_payment`) with bound parameters. The profile generalizes without change of method to a matrix of externalizations — card authorization, wire release, funds transfer, transaction posting, beneficiary change, limit increase, trade execution — each a discrete transition with its own actuation record; the single-action matrix is retained here because it is the one the enforcement-layer evidence covers. Consequence profile: C^*_{fin}. Externalization means: funds leave the account, the authorization response is transmitted, the instruction reaches the payment rail, or the ledger-changing transaction commits.

The upstream chain, made explicit. The register begins from a derived institutional control principle — **POL-TXN-001**: *transactions exceeding the institution's permitted risk conditions shall not be externalized unless all required authorization and escalation conditions have been satisfied* — stated as a derived case-study policy, not as any bank's wording.

Its register row, **RISK-TXN-001** (here B-01), is not "the model may classify fraud incorrectly"; it is "a transaction may externalize without all required execution-integrity conditions being satisfied" — an execution-authority risk, whose cause is incomplete mediation at the boundary, whose event is release without satisfied conditions, and whose consequences span financial loss, customer harm, sanctions/AML exposure, and irreversible downstream settlement. The control requirement, **CTRL-TXN-001**, is non-compensatory by construction: before release, every mandatory transaction-risk and execution-authority predicate applicable to the transaction shall be resolved and satisfied; an explicit prohibition present → DENY; a mandatory condition unavailable, stale, contradictory, or unresolved → HOLD; all mandatory conditions concurring → PERMIT. Under Ψ_K the requirement yields two structured controls that the DENY/HOLD distinction of Section V exists to keep apart:

```
# ACS-B01-02  (PROHIBIT -> DENY)
subject = TRANSACTION; action = RELEASE_TRANSACTION
condition: risk_class != PROHIBITED    (deterministic_lookup
     , active policy)
on_fail = DENY; on_unknown = HOLD
# ACS-B01-03  (REQUIRE -> HOLD)
subject = TRANSACTION; action = RELEASE_TRANSACTION
condition: enhanced_approval_present where risk_class ==
     ENHANCED_APPROVAL_REQUIRED
          (human_attestation; concurrence_policy per
     approval tier)
on_fail = DENY; on_unknown = HOLD    -> escalation:
     verification workflow
```

A known prohibited condition produces DENY; an absent required approval produces HOLD and enters the escalation route; neither externalizes. Φ then compiles the signed specifications into the predicate family of Table II and the invariant set, and the family is what an L-DREA-class authority evaluates at the transaction boundary. A fraud probability of 0.01 or a model confidence of 99.2% may contribute to a governed threshold contract if a policy explicitly consults it; it is never the permit.

What Case B does and does not demonstrate. B-04 illustrates consequence-based selection at its purest: at $s = 5$ — the bottom of the register, because a well-screened counterparty base makes the event rare — a sanctions breach is nonetheless statutory, material, and non-negotiable, and the gate's justification is its class, not its score. Each C^* row satisfies CV through a decisive gate on the sole authorized action path. However, because *every* Case B row carries a mandatory gate (four by C^*, two by other approved mandatory policy), any threshold $t \leq 5$ reproduces the gate set, so this register contains no Proposition 1 inversion or Proposition 2 collision and is not used by itself to establish $GD_{\min} > 0$. The separation results rest on Cases A and C; Case B contributes gate-source diversity, the explicit PROHIBIT/REQUIRE → DENY/HOLD pair, and the domain where enforcement-layer evidence exists.

The compiled Case B bundle (payload SHA-256 `2850155a2ee7d4c01db4748a891891bae27c48f4c9c2c7eccd30beb7d1aa33ce`) carries 9 predicate records — 6 risk-derived and 3 compiler invariants — against 6 obligation rows, with OPR $= 0/9$, PTC $= 9/9$, and $GD(15) = 4$ (B-03, B-04, B-05, and B-06 gated by class below the threshold). Representative rows of the correspondence are given in Table III. The failure-injection rows that exercise this family — thirteen injected conditions, each of which must produce no externalization — are summarized in Appendix D; the complete thirteen-predicate correspondence is Table IX there.

Evidence classification (stated plainly, as in prior work). The 284,807-event run reported in [15] over the public ULB credit-card dataset [41] (492 fraud-labeled rows mapped to should-deny, 284,315 genuine rows to should-permit) is a *golden-trace conformance* result — predicates pre-set from labels — demonstrating that compiled controls of these families evaluate deterministically at volume with zero false permits, zero false denials, zero unauthorized executions, complete hash-chain verification, and full replay determinism under the declared conditions; the run's figures are stated and bounded there and are not reproduced here. It is **not** detection evidence and is not presented as such. What Case B adds for *this* paper is upstream: the policy-to-register-to-disposition-to-predicate mapping that produced those predicate families, previously undocumented. The ULB labels remain the golden oracle for the downstream experiment; the upstream chain shows where the predicates it evaluated come from. The register above is a *forensic reconstruction*; Section XIII states this openly, and the mitigation is the hash-pinned artifact: once pinned, the mapping is reproducible going forward even though its origin is retrospective; the pinned Case B artifact is part of the release of Appendix C.

XI. Case Study C — Supervised-Autonomy Surgical Assistant (Cyber-Physical)

(Purpose: a third, structurally different domain — a discrete-transition authority over a physically actuating system that already carries an independent certified safety layer — and the register that exercises every cyber-physical schema element of GC-IR v1.1: safety-layer routing, transition admissibility, state binding, concurrence, standing permits, delegation, actor-scope revocation, advisory human-response, and safety-event correlation. Case is synthetic and standards-anchored: obligations resolve to ISO 14971, IEC 62304, IEC 80601-2-77, institutional privileging and consent policy, and manufacturer instructions-for-use of the kind publicly documented for robotically assisted platforms. No vendor's internal risk register, policy-compilation mechanism, execution-authority architecture, software, or telemetry is described, inferred, or claimed. Case C is presented as a fully specified design case*: every field required by Sections III–IV is populated and the specification (Steps 1–5 inputs, judgment record, dispositions, and specifications) is released with the artifact repository, but no compiled Case C bundle is measured in this paper; its register-level gate-divergence figures are properties of the printed tables, and compiled-artifact and conformance measurements are future work.)*

Profile highlights. Robotically assisted surgical platform with an AI assistance module. `S.autonomy_level` records a five-tier capability ladder (Table X) — data → insight → guidance → augmentation → supervised

TABLE II
CASE B: REGISTER → DISPOSITION → PREDICATE. ALL SIX ROWS RECEIVE RUNTIME DISPOSITIONS. B-01 YIELDS THREE SPECIFICATIONS (ONE RISK, SEVERAL PREDICATES — SECTION IV-D). FOR EVERY MANDATORY SPECIFICATION, `ON_UNKNOWN` = HOLD AND `ON_EVALUATION_TIMEOUT` = HOLD BY SCHEMA CONSTRAINT (APPENDIX A); `ON_FAIL` IS STATED PER SPECIFICATION.

risk_id	Event (observable)	consequence_class	$L \times I$	Gate (source)	Predicate sketch	on_unknown
B-01 unauthorized externalization	payment release without valid permit, under a prohibited risk class, or without required enhanced approval	C^*: authority	15	**mandatory** (C^*)	ACS-B01-01: valid single-use permit, state-bound to (txn hash, amount, beneficiary, account status, token status); ACS-B01-02: risk_class $\neq$ PROHIBITED (DENY); ACS-B01-03: required approval established (on_fail = DENY: denied or concurrence window expired; on_unknown = HOLD: pending or unavailable)	HOLD
B-02 threshold breach	amount $>$ per-transaction limit / transaction outside actor authority	financial_loss	16	mandatory (other)	two specifications: ACS-B02-01 (hard authority ceiling) amount $\leq$ absolute_authority_ceiling, on_fail = DENY; ACS-B02-02 (delegated escalation threshold) amount $\leq$ ordinary_limit $\vee$ additional_approval_established; on_fail = DENY (approval denied or concurrence window expired), on_unknown = HOLD (approval pending or unavailable) $\rightarrow$ additional-authorization route, PERMIT once established (threshold contracts on both bounds)	HOLD
B-03 velocity anomaly	txn count/window $>$ bound	financial_loss	12	mandatory (other)	window count $\leq$ approved bound	HOLD
B-04 sanctions/restricted counterparty	counterparty $\in$ restricted list, or screening artifact unavailable	C^*: statutory (material)	5 (L=1, I=5)	**mandatory** (C^*)	signed screening artifact reports counterparty $\notin$ restricted list; artifact absent or stale $\rightarrow$ HOLD, never inferred clear	HOLD + escalate
B-05 replay/duplicate	permit reuse or duplicate txn hash	C^*: irreversible	10	**mandatory** (C^*)	permit single-use; txn hash unseen; permit scope matches the externalized action	HOLD
B-06 evidence failure	receipt not committed between authorization and actuation	C^*: statutory (material)	10	**mandatory** (C^*; **coincides with INV-EVIDENCE-COMMIT**)	authorization-then-commit-then-actuate receipt present & chained	HOLD

autonomy — the deployed tier being *supervised autonomy for bounded subtasks*: the module may propose and, once authorized, execute a discrete subtask under an identifiable supervising surgeon; it may never remove the surgeon from the loop, and surgeon control and accountability are preserved as a profile-level principle, matching the publicly stated design intent of contemporary platforms [44]. `S.architecture.safety_layers` declares an independent runtime safety layer — collision and force envelope, instrument interlocks, emergency stop, fallback-to-safe-pose — with standing authorization INV-SAFETY-RECOVERY; `S.architecture.control_loops` declares one (`control_loop_ref`, `control_loop_period`, `excluded_action_family`) entry per excluded family — the force-scaling/haptic loop, the envelope-keeping loop, and the teleoperation tracking loop — so that AD-1 exclusions cite the loop they belong to rather than a single global period. The authority matrix admits discrete transitions only: `ACTIVATE_INSTRUMENT`, `ENERGY_ACTIVATE`, `FIRE_STAPLER`, `ENTER_AUTONOMOUS_SUBTASK`, `TRANSFER_CONTROL`, `DELEGATE`, `PRESENT_GUIDANCE`, `EXPORT_DATA`. Continuous teleoperation, force scaling, and envelope-keeping are excluded under AD-1 with the exclusion recorded in `S.exclusions`. Human actors: supervising surgeon (privileged for the procedure class), second surgeon (remote or bedside), circulating nurse (attester for data export). Obligations (Step 3) resolve to ISO 14971, IEC 62304, IEC 80601-2-77, manufacturer IFU contraindications, institutional privileging, and consent scope, with the `requirement_class` of each and the runtime context the predicates consume listed in Appendix D. Consequence profile: C^*_{cp}. Declared heat-map rule: the institution's clinical-risk matrix gates at $T_H = 15$. Figure 4 gives the ordered composition of authorization and safety for this profile, Table IV the outcome matrix over the two conditions, and Tables V–VI the register.

Why this profile and not a finance-shaped one. Every finance row's hazard crosses the execution boundary once, at a single externalization. Here the hazards are of four kinds that the financial cases cannot exhibit: a physical act with a bounded but real reversibility profile; a continuous process that the authorization layer must *not* mediate; a transfer of the answerable party; and a data-plane disclosure from inside the operating room. The register is built to place at least one row in each.

Dispositions: 13 runtime, 3 non-runtime, 0 accepted, 0 unresolved — DC = 1 by construction once dispositions are signed; RCY = 13/16 (descriptive). Every C^*_{cp} row satisfies CV through at least one decisive mandatory gate on its authorized hazardous action path; supporting conditions are recorded with `mandatory_role = supporting`.

C-01 note (the contraindication as a lookup, not a

TABLE III
CASE B: REPRESENTATIVE ROWS OF THE THIRTEEN-PREDICATE CORRESPONDENCE (FULL TABLE IN APPENDIX D). NAMES ARE CASE B'S DOMAIN-LEVEL RENDERING OF THE CHECK-LEVEL PREDICATES OF [15]; THE MAPPING TO ARTIFACT IDENTIFIERS IS RECORDED IN THE RELEASE MANIFEST.

Predicate	Origin	Failure outcome
P3 transaction_within_authority	B-02 ACS-B02-01 (ceiling) / ACS-B02-02 (escalation)	DENY above absolute ceiling; above ordinary limit: HOLD while approval pending or unavailable, DENY if denied or window expired
P5 sanctions_clear	B-04	DENY / HOLD (service unavailable)
P7 state_digest_match	INV-STATE-BINDING (digest at the boundary equals digest at authorization; snapshot freshness is a threshold contract on the bound attributes)	HOLD
P8 required_approval_present	B-01 ACS-B01-03	HOLD while pending or unavailable; DENY if denied or concurrence window expired; evaluator timeout HOLD
P11 policy_version_valid	INV-VERSION	HOLD

TABLE IV
OUTCOME MATRIX OVER THE TWO INDEPENDENTLY EVALUATED CONDITIONS. PASS DENOTES SAFETY-LAYER NON-INTERVENTION (ADMISSIBILITY UNDER CONTINUOUS SUPERVISION OVER THE TRANSITION INTERVAL), NOT A ONE-SHOT SAFETY PREDICATE; FAIL DENOTES INTERVENTION. ONLY THE FIRST CELL IS ELIGIBLE TO ACTUATE; THE SECOND IS THE EXPERIMENTALLY DECISIVE ONE.

Authorization	Safety layer (non-intervention)	Outcome
PERMIT	PASS	eligible to actuate; external effect may occur
DENY	PASS	no external effect — governance/authority veto; the class the safety envelope does not identify
HOLD	PASS	no external effect — valid permit not established
PERMIT	FAIL	no external effect — safety veto; PERMIT does not force execution
DENY	FAIL	no external effect
HOLD	FAIL	no external effect

judgment). A manufacturer's published contraindication of an instrument class for a task in a procedure class is the cleanest possible refinement input: it already names the mechanism (the instrument), the event (activation for the task), and the consequence (harm requiring intervention). The row is generic: it names an instrument class, a task, and a procedure class, not a product, and asserts nothing about any manufacturer's instructions, notices, or internal controls. The observable is the triple (`procedure_class`, `instrument_class`, `task`) resolved against a versioned contraindication table whose hash is in the bundle; the predicate is a deterministic lookup exactly as R-05's restricted-list check is, and

fig4_composition

Fig. 4. Ordered composition of authorization and safety. The authorization gate decides whether a discrete transition may be attempted and commits its receipt before actuation; the runtime safety layer supervises the resulting motion with standing authority and never asks permission. A safety intervention is logged as a correlated `SafetyEvent`, and the interrupted transition returns HOLD through INV-STATE-BINDING rather than resuming on its old permit.

its structured control reads `procedure` $\in$ {contraindicated set} $\wedge$ `instrument` = class $\wedge$ `task` = task $\Rightarrow$ PROHIBIT, compiled to `contraindication_absent` $:= \neg(\cdots)$ with `on_fail = DENY`. The warrant boundary is stated in the ACS: `procedure_class` is the value declared by the scheduling system of record, not a value inferred from video — that inference would be RC-03 and lives upstream in the evidence-production layer. If the declared value is absent or inconsistent, the outcome is HOLD, never a silent permit of the instrument. The decisive point is that this row is not trajectory safety: with trajectory, collision, force, velocity, and controller health all passing, the transition is still DENY, and the safety layer is neither deficient nor consulted — the two mechanisms evaluate different classes of condition.

C-03 / C-04 note (authority and its transfer). C-03 is the supervised-autonomy handoff at its sharpest: an assistance module may report 99.7% confidence that the next action is correct, and confidence is not authority. The conditions — authenticated supervisor, privileged for the procedure class, holding the control token, in supervised mode, capability admitted for this step, instrument configuration valid, `input_provenance` complete as declared, plus the version and freshness invariants — must all hold, and the safety layer's non-inhibition is deliberately not among them, because the safety layer is not consulted by the gate; it supervises after the gate, with veto. C-04 is where single-signer attestation fails: a handoff has two answerable parties, and a permit signed by only one records an accountability gap rather than closing it. The 2-of-2 concur-

TABLE V

CASE C: REGISTER → DISPOSITION → PREDICATE, ROWS C-01–C-08. THIRTEEN OF SIXTEEN ROWS RECEIVE RUNTIME DISPOSITIONS; THREE RECEIVE NON-RUNTIME DISPOSITIONS. $L \times I$ VALUES ARE RESIDUAL RATINGS. GATE SOURCE C^* DENOTES C^*_{cp}. CONTINUED IN TABLE VI.

risk_id	Event (phenomenon → approved observable)	consequence_class	$L \times I$	Disposition / gate (source)	Predicate sketch	on_unknown
C-01 contraindicated instrument–procedure combination	instrument of a contraindicated class activated for a contraindicated task in a contraindicated procedure class (IFU)	C^*: physical_harm (requires_ intervention)	8 (L2·I4)	runtime · **mandatory** (C^*)	(procedure_class, instrument_class, task) $\notin$ versioned IFU contraindication table (lookup; table hash in bundle)	HOLD
C-02 energy delivered outside authorized state	`ENERGY_ACTIVATE` proposed under a permit whose state digest no longer matches	C^*: physical_harm (irreversible)	10 (L2·I5)	runtime · **mandatory** (C^***; coincides with INV-STATE-BINDING**)	state_binding = (procedure_step, instrument_id, port_config, patient_position_hash); digest at boundary == digest at authorization; permit single-use	HOLD
C-03 consequential autonomous actuation without supervisory authority	`FIRE_STAPLER` / `ENTER_AUTONOMOUS_SUBTASK` proposed without identifiable authorized supervisor	C^*: authority	5 (L1·I5)	runtime · **mandatory** (C^*)	supervisor.authenticated $\wedge$ supervisor $\in$ privileged(procedure_class) $\wedge$ control_token.owner == supervisor $\wedge$ autonomy_mode == SUPERVISED $\wedge$ capability $\in$ authority_matrix(procedure_class, step) $\wedge$ instrument config valid $\wedge$ input_provenance complete	HOLD
C-04 control transfer without concurrence	`TRANSFER_CONTROL` between consoles or into autonomy mode without both parties' attestation	C^*: accountability_transfer	8 (L2·I4)	runtime · **mandatory** (C^*)	concurrence_policy 2-of-2 (releasing, receiving; distinct actors; roles = privileged surgeon), all-or-nothing, window w_{tc}; concurrence_expiry_response = DENY; on_evaluation_timeout = HOLD	HOLD
C-05 actuation after supervisor withdraws authority	actuation record whose actor–scope carries an ActorScopeRevocation with effective_time $\leq$ authorization_time	C^*: authority	5 (L1·I5)	runtime · **mandatory** (C^*)	$\neg$RevokedScope(REG, supervisor, action_tuple, t_{auth}); cascade voids delegated child permits	HOLD
C-06 force-scaling / haptic loop error causing excess tissue force	control-loop-rate correction exceeds force bound	patient_harm (requires_ intervention)	16 (L4·I4)	**nonruntime · RC-06 → runtime_safety**	— (fails AD-1: feedback primitive of control_loop_ref = haptic/force-scaling loop; supervised by force envelope under INV-SAFETY-RECOVERY)	—
C-07 collision / envelope excursion	instrument leaves geometric or force envelope	patient_harm (requires_ intervention)	12 (L3·I4)	**nonruntime · RC-06 → runtime_safety**	— (interlock stop logged as SafetyEvent; interrupted transition returns HOLD via INV-STATE-BINDING)	—
C-08 contraindicated or stale guidance presented	`PRESENT_GUIDANCE` proposes a recommendation contraindicated for the procedure class or produced against a stale context snapshot	patient_harm (reversible)	9 (L3·I3)	runtime · mandatory (other: institutional clinical-safety policy) + HumanResponse	guidance_class $\notin$ contraindication table $\wedge$ context_snapshot_age $\leq$ bound (threshold contract); response $\in$ {followed, deviated} logged against permit_id	HOLD

rence policy is all-or-nothing with deny-on-expiry: a releasing attestation without a receiving one when the window closes is DENY (`concurrence_expiry_response`), not a pending transfer — whereas the evaluator itself failing to finish inside its latency bound is HOLD (`on_evaluation_timeout`); the two are different events with different responses. A request from a surgeon who is authenticated but is not the current control owner is DENY where ownership is resolved and HOLD where a transfer is in flight.

C-06 / C-07 note (what the gate must not touch). Both rows score at or above the declared threshold and both are ungated at the authorization layer — correctly. A force-scaling correction is a feedback primitive of a declared control loop (`control_loop_ref` in `S.architecture.control_loops`); placing it behind a permission gate would put an event-rate authority inside a control-loop-rate path it is not certified for and would make the gate the binding latency of the loop. AD-1 rejects the tuple, RC-06 records why, and the disposition routes the hazard to the `runtime_safety` family under INV-SAFETY-RECOVERY. The interlock stop of C-07 is a SafetyEvent with standing origin; the transition it interrupted does not resume on its old permit, because the state digest has changed. This is the composition of Section II made concrete: the gate decides whether a transition may be attempted, the safety layer decides whether the resulting motion may continue, and each is visible to the other only through the evidence chain.

The remaining row notes (C-02, C-05, C-08/C-13, C-09, C-11/C-12, C-14/C-15), the E1–E19 failure-injection matrix, and the planned conformance harness are given in Appendix D; the capability ladder recorded in `S.autonomy_level` is Table X there.

Reading of Propositions 1 and 2 on this register (Figure 5)**.** C-01 ($s = 8$), C-03 (5), C-04 (8), C-05 (5), C-14 (6), and C-15 (6) sit at the bottom of the heat map yet carry mandatory gates; C-06 (16) sits at the top and carries none. The inversion ($s_{\text{C-01}} = 8 < 16 = s_{\text{C-06}}$, $g_{\text{C-01}} = 1$, $g_{\text{C-06}} = 0$) satisfies Proposition 1, so $GD_{\min} \geq 1$ on this register. The four-way collision at $s = 12$ (C-07, C-09, C-11, C-16 with gates 0, 1, 0, 0) instantiates Proposition 2. Computed on the register as specified in Tables V–VI, against the institution's declared $T_H = 15$: $GD(15) = 12$ — eleven C^* and other-mandatory rows are gated below the threshold, and C-06 is

TABLE VI
CASE C: REGISTER → DISPOSITION → PREDICATE, ROWS C-09–C-16 (CONTINUED FROM TABLE V).

risk_id	Event (phenomenon → approved observable)	consequence_class	$L \times I$	Disposition / gate (source)	Predicate sketch	on_unknown
C-09 intraoperative data disclosed outside consent scope	EXPORT_DATA of video/telemetry to a destination or purpose not in the consent record	C^*: protected_data_disclosure	12 (L3·I4)	runtime · **mandatory** (C^*)	signed consent artifact covers (data_class, destination, purpose); attester = circulating nurse (single-signer attestation)	HOLD + escalate
C-10 insight from a model outside its intended use	recommendation or phase detection from a model not validated for this procedure class	patient_harm (reversible)	9 (L3·I3)	runtime · mandatory (other)	model_id ∈ validated_models(procedure_class) ∧ model_version == M.version_binding (coincides with INV-VERSION)	HOLD
C-11 attestation fatigue on routine steps	per-step attestation dwell outside band for pre-authorized step classes	governance	12 (L3·I4)	runtime · advisory → monitoring	attestation-time telemetry within band for transitions covered by StandingPermit; deterministic proxy for substantive review	warn
C-12 delegated subtask exceeds delegation scope	child permit issued by DELEGATE wider than parent in scope, validity, or action set	C^*: accountability_transfer	10 (L2·I5)	runtime · **mandatory** (C^***; coincides with INV-DELEGATION-NARROWING**)	(child.scope, validity, actions) ⊊ (parent.scope, validity, actions) ∧ child.parent_permit_id resolves	HOLD
C-13 stale, wrong-patient, or unprovenanced input data	context snapshot fails identity, freshness, or lineage check	patient_harm (requires_intervention)	15 (L3·I5)	runtime · mandatory (other)	patient_id == scheduled_patient ∧ snapshot_age ≤ bound ∧ lineage_hash chain verifies; defect propagates to C-03 via input_provenance	HOLD
C-14 authorization evaluation exceeds latency bound	governor fails to complete evaluation within evaluation_latency_bound	governance (availability)	6 (L2·I3)	runtime · mandatory (other; governance row over INV-LATENCY, as R-11 over INV-VERSION)	independent deadline watchdog (outside the evaluator) expires before evaluation-complete ⇒ HOLD; no partial permit; bound under threshold contract; phase PRE_AUTHORIZATION	HOLD
C-15 unrecorded safety intervention / incomplete recovery evidence	interlock or fallback fires and no signed SafetyEvent with standing origin and correlated permit/bundle is recorded within Δ_{se}	governance (evidence)	6 (L2·I3)	runtime · mandatory (other; runtime evidence invariant over INV-SAFETY-RECOVERY; phase POST_EVENT_AUDIT)	SafetyIntervention ⇒ $\Diamond_{\leq \Delta_{\mathrm{se}}}$ SignedSafetyEvent; MissingSafetyCorrelation ⇒ HOLD on the next ordinary authorization in scope; never gates the intervention itself	HOLD
C-16 degraded supervisor performance (fatigue, distraction)	—	patient_harm	12 (L3·I4)	**nonruntime · RC-03 → human_process**	— (no approved alertness observable, evidence producer, or threshold contract; scheduling and relief policy)	—

ungated above it — and $GD_{\min} = 4$, attained at $t = 5$ — the minimum observed score, so that within $\mathcal{T}$ every threshold at or below it is equivalent and gates every row; the four residual misclassifications under that best scalar rule are exactly C-06, C-07, C-11, and C-16, the continuous-control, safety-layer, monitoring, and human-process rows that must *not* be gated at the authorization boundary. The surgical register therefore sharpens the finance result: in a domain where consequential events are rare by design, no plausible heat-map threshold reproduces the rows required by the approved consequence-class policy, and the best-fitting one gates rows the safety architecture forbids gating. Both sums run over all sixteen rows; the figures are properties of Tables V–VI and are recomputable from them.

Non-runtime dispositions (three rows, two reason codes). C-06 and C-07 — **RC-06: continuous control** — are routed to the `runtime_safety` family, naming the force envelope and interlock in `S.architecture.safety_layers` and the standing-authorization invariant under which they act. C-16 — **RC-03: requires probabilistic judgment** — is routed to a human-process control (scheduling, relief, and fatigue policy), for the same reason R-15's "tone" was: no alertness observable, evidence producer, or threshold contract has been proposed to or approved by the governance forum. A fatigue classifier's score could be evaluated deterministically under commitment 2 of Section V once such a contract exists; the obstacle is the absence of an approved operationalization, not a categorical bar on probabilistic evidence.

What Case C does and does not demonstrate. Case C demonstrates *representational* applicability — that the schema, the admissibility rule, and the coverage rule express a cyber-physical authority matrix and its hazards without loss — and not *operational* applicability, which requires the compiled bundle and the conformance harness of Appendix D and is deferred. Within that boundary it demonstrates that GC-IR v1.1 expresses a cyber-physical authority matrix without borrowing the safety layer's vocabulary or mediating its loop, that the DENY/HOLD/PERMIT semantics carry across a physical actuation boundary unchanged, and that the coverage rule yields an assignment not representable by the scalar heat-map score alone — the rows required by the approved consequence-class policy. It does not demonstrate trajectory safety, clinical benefit, or conformance of any product; those are the safety layer's and the manufacturer's to show under the cited standards, and the claim boundary (Section XV) says so.

XII. EVALUATION AND STATISTICAL ANALYSIS PLAN

Scope of claims. All empirical results reported by this paper are claim class C1 or C2 (Section XII-G); Theorem 1 and Propositions 1–3 are formal results, and the Case C register-level quantities are deterministic recomputations over printed rows — neither is an evidence class and neither is assigned one. RQ5 — the comparative expert study — is **preregistered and deferred**: its full protocol is frozen and

Fig. 5. Case C gate assignment against residual rating. Seven C^*_{cp} rows and four other-mandatory rows are gated below the declared threshold $T_H = 15$ (C-13, other-mandatory at $s = 15$, is gated by both rules), and the highest-scoring row (C-06, continuous control) is correctly ungated at the authorization layer although the threshold rule would gate it: $GD(15) = 11+1 = 12$, $GD_{\min} = 4$.

publicly hash-committed with this paper's artifact release, and it is executed and reported as an independent follow-up. No comparative-superiority claim over unaided manual practice is made in this paper; the method-and-artifact claims (C1–C5 of Section I-C) rest on the formal results, the executable compiler, and the measured determinism, disposition, gate-divergence, and traceability properties. Case C is a fully specified design case: its register-level GD and $GD_{\min}$ figures are properties of Tables V–VI and are stated in Section XI; no compiled Case C bundle is measured here, and the compiled-artifact metrics (DC, OPR, ODC, PTC, CV, TD, Monte Carlo) are reported for Cases A and B only.

A. Evaluation questions

- **RQ1 — Total disposition:** Does every risk receive exactly one disposition (runtime, non-runtime, accepted, or unresolved), and does a release-admissible artifact contain no unresolved disposition?
- **RQ2 — Determinism:** Do semantically equivalent inputs produce the same canonical payload hash across supported environments?
- **RQ3 — Gate selection:** Does consequence-class coverage differ from the predeclared heat-map comparator, can any scalar threshold reproduce the approved gate assignment, and does the divergence persist in a cyber-physical register where consequential events are rare by design?
- **RQ4 — Traceability:** Are obligations, dispositions, predicates, receipts, safety events, and human responses connected without orphan or temporally invalid links?
- **RQ5 — Human translation quality (preregistered, deferred):** Compared with unaided manual practice, does the structured refinement method reduce silent narrowing, omissions, orphan controls, and inconsistent gate assignments? Answered by the committed follow-up study, not by this paper.

B. Primary metrics

Every measured value in Table VII is defined once in the manuscript source and is recomputed from the tagged release by a check script shipped with the artifacts (Appendix C), so that no reported figure exists only as manuscript arithmetic. RCY is descriptive, not a quality target; forcing it toward 1 would incorrectly encourage non-runtime risks to be translated into unsuitable predicates. Threshold, operator, authority-tuple, and unknown-handling fields require exact agreement with the adjudicated reference; semantic similarity alone is not accepted.

C. Independent reference standard

Under the deferred protocol, the reference standard will be produced by an independent three-person adjudication panel with expertise spanning AI governance, operational risk, and policy/control engineering. Panel members have no authorship, ownership, or commercial interest in the method. The panel first assesses each held-out row independently, then conducts a blinded, structured Delphi reconciliation; disagreements and final rationales are retained. The author does not participate in adjudication and does not modify the held-out register, catalog, or thresholds after panel review begins. Φ output is not treated as ground truth; it is compared against the independently adjudicated Approved Control Specifications.

D. Comparative expert study (frozen protocol; deferred execution)

The RQ5 protocol is frozen, hash-committed with the artifact release, and executed as an independent follow-up. In summary: at least eight (at most twelve) independent practitioners map a held-out register of at least 30 rows across three domains, including one non-financial domain, in a randomized counterbalanced crossover of unaided-manual versus GC-IR/K-assisted conditions with no row seen by a reviewer in both; the reference standard is produced by an independent three-person adjudication panel (Section XII-C); primary outcomes are the silent-narrowing rate, incorrect-gate rate, and disposition completeness; agreement is reported with Fleiss' κ [32], Krippendorff's α [33], and Gwet's AC1 [34] with cluster-bootstrap intervals; and defect outcomes are analyzed with a hierarchical Bayesian logistic model with reviewer and row effects. Recruitment is precision-based (minimum eight reviewers, continuing to twelve if the 95% interval for the primary agreement estimate has half-width above 0.15 after eight — an operational rule, not a power guarantee), and with eight to twelve reviewers over about thirty rows the hierarchical posterior will be broad and is reported

TABLE VII
METRIC DEFINITIONS AND RESULTS. CASES A AND B ARE MEASURED ON THE PINNED ARTIFACTS; CASE C IS A DESIGN CASE WHOSE REGISTER-LEVEL GATE-DIVERGENCE FIGURES ARE STATED FROM TABLES V–VI; ITS COMPILED-ARTIFACT METRICS ARE OUT OF SCOPE FOR THIS PAPER.

Metric	Definition	Target/Result
Disposition Completeness (DC)	$\lvert\{r_i : \Delta_i \in (\text{runtime}, \text{nonruntime}, \text{accepted})\}\rvert \div \lvert R\rvert$	Case A: 1.000 (16/16); Case B: 1.000 (6/6) — both release-admissible; Case C 16/16 by construction on the printed register (design case; not compiled)
Runtime Compilability Yield (RCY)	$\lvert\{r_i : \Delta_i = \text{runtime}(D_i)\}\rvert \div \lvert R\rvert$	Case A: 0.8125 (13/16); Case B: 1.000 (6/6) — descriptive, not a quality target (Case C register: 13/16 by construction)
Non-Runtime Disposition Rate (NDR)	$\lvert\{r_i : \Delta_i \in (\text{nonruntime}, \text{accepted})\}\rvert \div \lvert R\rvert$	Case A: 0.1875 (3/16); Case B: 0.000 (0/6) — descriptive
Orphan Predicate Rate (OPR)	$\lvert\{p \in P : \text{origin}(p) \notin R \cup \text{INV}\}\rvert \div \lvert P\rvert$	Case A: 0.000 (0/19); Case B: 0.000 (0/9) — required 0
Obligation Disposition Completeness (ODC)	$\lvert\{o \in O : o \rightsquigarrow \text{runtime} \vee \text{nonruntime} \vee \text{accepted}\}\rvert \div \lvert O\rvert$	Case A: 1.000 (12/12); Case B: 1.000 (6/6)
Predicate Traceability Completeness (PTC)	$\lvert\{p \in P : p \rightsquigarrow \text{risk} \vee \text{compiler invariant}\}\rvert \div \lvert P\rvert$	Case A: 1.000 (19/19); Case B: 1.000 (9/9)
C^* Coverage (CV)	$\lvert\{r_i : C^*(c_i) =$ $1 \wedge$ each hazardous action path mandatorily gated$\}\rvert \div \lvert\{r_i :$ $C^*(c_i) = 1\}\rvert$	1.000 (Case A 6/6; Case B 4/4) — required 1
Translation Determinism (TD)	$\sum_k \mathbf{1}[H_k = H_{\text{reference}}] \div N$ over environments and permutations	1.000 (31/31 runs per case, 62/62 executions, Cases A–B; reference hashes reproduced on 8 environments across three operating systems; run design in Section XII-F) · Case C not compiled
Gate Divergence	$GD(T_H)$ with predeclared frozen T_H; and $GD_{\min}$	Case A: $GD(15) = 3$, $GD_{\min} = 3$ · Case B: $GD(15) = 4$, $GD_{\min} = 0$ · Case C: $GD(15) = 12$, $GD_{\min} = 4$ (register as specified)
Silent-Narrowing Rate (SNR)	$\sum_i SN_i \div \lvert R\rvert$ against the adjudicated reference	primary outcome of the deferred RQ5 study; not reported here
Field-Level Derivation Fidelity (DF)	$DF_i = \lvert F_i \cap F_i^*\rvert \div \lvert F_i^*\rvert$; DF = mean over R	primary outcome of the deferred RQ5 study; not reported here

as such. The full protocol — block construction, recorded fields, the optional exploratory pilot, the reliability statistics, the Bayesian model with its priors and convergence criteria, and the stopping rule — is the hash-committed preregistration bundle of the artifact release (Appendix C). It is evaluation design, not evidence: nothing in this paper's results depends on it.

E. Monte Carlo rating-robustness analysis

Monte Carlo analysis measures the sensitivity of heat-map gate selection to rating uncertainty; it is not a substitute for Propositions 1–2. For each risk r_i, discrete probability masses over plausible ratings are frozen before sampling and released with the artifact — for the pinned Case A run reported here, frozen by the author; panel-frozen masses belong to the deferred protocol (Section XII-C): $L_i^{(k)} \sim \text{Cat}(\pi_{L_i})$, $I_i^{(k)} \sim \text{Cat}(\pi_{I_i})$. For $K \geq 250{,}000$ draws, $s_i^{(k)} = L_i^{(k)} I_i^{(k)}$ and $h_i^{(k)} = \mathbf{1}[s_i^{(k)} \geq T_H]$. Let $h_i^{(0)} = \mathbf{1}[s_i^{(0)} \geq T_H]$ be the heat-map decision at the nominal (approved) rating. The heat-map gate-flip probability is $FP_i^{\text{heat}} = (1/K)\sum_k \mathbf{1}[h_i^{(k)} \neq h_i^{(0)}]$ — the probability that rating uncertainty alone changes the heat-map decision for row i; disagreement with the *approved* assignment is a different quantity and is reported as the distribution of $GD(T_H)$ over draws. Conditional on unchanged consequence classification and policy version, the corresponding class-based flip probability is $FP_i^{C^*} = 0$ (Theorem 1(d)). Reported quantities: per-risk flip probability; expected gate changes per register; distributions of $GD(T_H)$ and $GD_{\min}$; and Monte Carlo standard error $\text{MCSE}(\hat{p}) = \sqrt{\hat{p}(1-\hat{p})/K}$, bounded above by 0.001 at $K = 250{,}000$. Measured on Case A at $K = 250{,}000$ (seed 20260201; measured MCSE ≤ 0.00093): the maximum per-risk heat-map flip probability is 0.321 (R-06, at $s = T_H$), concentrated on rows adjacent to T_H; on Case B under the same design it is 0.318 (B-01, likewise at T_H); in both cases $FP_i^{C^*} = 0.000$ across the register by construction, with zero C^* changes observed in 1,000 probe draws per case — a property of the policy definition under frozen classification, not an empirical finding of superiority. Case C is a design case and was not subjected to Monte Carlo analysis (Section XI).

F. Determinism and adversarial compiler testing

TD testing includes: repeated runs; key and row reordering; equivalent numeric encodings rejected or normalized per schema; locale and time-zone changes; supported operating systems or reproducible containers; malformed references; duplicate IDs; conflicting predicates; expired approvals; stale catalog versions; missing evidence producers; unknown consequence classes; unauthorized action triples; C^* rows with missing mandatory coverage; attempted in-place payload mutation (must fail the hash check); post-retirement bundle use against the lifecycle registry; the Case B failure-injection rows (Section X); and the Case C matrix E1–E19 with its compile-time injections (Section XI). Property-based tests verify total disposition, authority closure, transition admissibility, mandatory unknown- and timeout-failure, state-binding requirement on C^* transitions, concurrence all-or-nothing, delegation narrowing, C^* coverage, origin closure, payload immutability, and temporal receipt validity. Measured: TD = 1.000 over 31 compilation runs per case for Cases A and B, executed inside the pinned reproducible container, the 31 comprising 10 repeated baseline runs, 10 row-order permutations, 5 key-order permutations, 3 locale variations, and 3 time-zone variations, as recorded in the run manifest of the tagged release (Appendix C); 62 executions in total,

62 passed. The committed reference hashes were additionally reproduced on 8 independently installed environments — Linux, macOS, and Windows hosts on x86-64 and 64-bit ARM (reported as arm64 or aarch64 by host) under CPython 3.11 and 3.12, plus the pinned container — each yielding TD = 1.000 with matching payload hashes. This is determinism across the tested supported environments; it is not a claim of platform independence beyond them (Section XIII).

The adversarial and validation suite of the same release comprises 62 scenarios (54 negative, 8 positive controls, no code mismatches), 26 structural checks, 16 property-based tests over 1,427 generated examples, 18/18 negative controls fired, and the thirteen Case B injections, each producing no externalization at the consuming authority. Of the ten audit queries of Appendix A, 6 execute over the released Case A and Case B evidence stores and return empty result sets; the remaining four (authorized-actuation ordering, revoked actor scope, safety-event correlation, and human-response coverage) require actuation, revocation, safety-event, and human-response fixtures and are exercised as compile-time and fixture checks rather than as queries over a compiled evidence store in this release.

G. Preregistration and stopping rules

Before execution, every element the study depends on — hypotheses and primary outcomes, case artifacts and held-out-register hash, derivation catalog, adjudication instrument, thresholds, both C^* profiles, the admissibility declarations, compiler commit and container digest, seeds, priors, Monte Carlo distributions, exclusion rules, and analysis code — is publicly hash-committed. The deferral of RQ5 is itself part of the preregistration: the public commitment binds the comparative study to be executed as frozen, so the deferral is cryptographically committed rather than promised. Primary outcomes of the deferred study are SNR, incorrect-gate rate, and DC; other outcomes are secondary, and no conclusion is based solely on an uncorrected collection of exploratory comparisons.

Reporting discipline. Every published result carries a claim class and is stated only at the class its evidence supports: C1 (author-operated measurement under declared conditions) or C2 (conformance run against a declared, versioned profile with reported bounds). No C3 (independently verified) or C4 (consensus-bound) claim is made anywhere in this paper. The internal selection of frozen elements is not a freeze — the public timestamped commitment is.

Downstream interface (out of scope here). Once a compiled bundle is deployed, its runtime performance is measured at the enforcement layer by the six primary metrics of [15] (Section VIII-G) and its decision-path latency reporting (Table 19); this paper declares the latency bound in the ACS and does not measure it. This paper's metrics stop at compilation; the two families meet at the compiled-bundle hash, and conformant receipts carry the requirement reference this paper specifies. **Deliberately not evaluated:** detection performance, production impact, false-denial rates in live traffic, trajectory safety — those require the shadow-mode deployment and the conformance harness identified as future work and belong to a subsequent paper.

XIII. Limitations and Threats to Validity

1) **Case provenance.** Case A is synthetic (fully specified, hash-pinned, reproducible — not a deployed system). Case B's runtime evidence is conformance-class [15], and its register-to-predicate mapping is a forensic reconstruction, stated openly, with the hash-pinned artifact as mitigation. Case C is synthetic and standards-anchored: its obligations, contraindication table, and safety-layer declaration are specified from public standards and publicly documented instructions-for-use of the kind common to robotically assisted platforms; no vendor's internal register, software, or telemetry is described; and it is a design case — not compiled into a measured bundle, not run against a simulator or hardware (Section XI).
2) **Single-author methodology risk.** The independent adjudication panel and the preregistered comparative expert study are the mitigation and must actually be executed.
3) **Obligation determination remains human.** Step 3 is a legal function; the method inherits any error in it (a garbage-in bound stated explicitly), though the expanded obligation record makes the determination auditable and legal-source version drift detectable.
4) **Semantic-refinement dependence.** Φ is deterministic only after Approved Control Specifications have been governed and signed; Ψ_K is single-valued only given the signed judgment record J. No deterministic interpretation of unconstrained prose is claimed.
5) C^* **membership is a governance judgment.** The method makes it explicit, materiality-qualified, and auditable — not automatic; Theorem 1 establishes that the score is informationally insufficient for the approved assignment, not that any particular assignment is substantively correct.
6) **Generality is representational, not operational.** A second consequence profile C^*_{cp} and a fully specified cyber-physical register (Case C) show that the schema expresses a physically actuating domain without loss, and the non-financial rows of the held-out instrument partially test it; no Case C bundle is compiled or exercised here, and nothing is demonstrated in deployment or against clinical outcomes.
7) **Cyber-physical composition assumed, not certified.** The method declares the safety layer, routes continuous-control hazards to it, and correlates its interventions in the evidence chain; it does not verify the safety layer's correctness, coverage, or certification, and nothing here substitutes for IEC 80601-2-77 or ISO 14971 conformance. AD-1's timing condition is only as good as `S.architecture.control_loops`: a misdeclared `control_loop_period`, or an action family assigned to the wrong `control_loop_ref`, could admit a feedback primitive into the matrix, and the semantic condition is a governed classification that can

also be wrong; the declarations are governed profile fields and reassessment triggers, which is the mitigation available at compile time.

8) **Authorization-soundness and epistemic bounding, not semantic correctness.** Compilation warrants that the predicate set faithfully and deterministically encodes the *approved* specifications — not that the specifications are complete over the harm space, nor that a permitted action is the right action in the world; PERMIT means the authority found no prohibition under the predicates it evaluates. Compiled predicates evaluate the evidence declared to them, not the reality that evidence purports to describe; authentic-but-false upstream evidence lies outside any predicate's warrant. The mitigation is declaration (the ACS evidence-semantics and warrant-boundary statements) plus complementary controls, not a claim of closure; conformance claims cannot be read as safety guarantees.
9) **Reference-standard and catalog uncertainty.** The adjudicated reference remains expert judgment rather than objective ground truth; independent review, retained disagreement, and uncertainty reporting mitigate but do not eliminate this. A derivation catalog may encode omissions or institutional bias, so catalog versions, approvers, and changes are governed evidence evaluated independently. Kappa may be unstable when one disposition dominates; Krippendorff's α, Gwet's AC1, category-specific agreement, and raw confusion matrices are reported alongside it.
10) **Ordinal ratings and Monte Carlo scope.** Likelihood and impact scales are ordinal; products and continuous sensitivity bounds are used only to analyze the enterprise heat-map practice, not to claim interval-scale measurement. Rating perturbation estimates sensitivity under the frozen input distributions released with the artifact, not real-world event frequencies or harm probabilities.
11) **Temporal evidence dependence.** Receipt validity requires trusted time, trustworthy version and evidence producers, and an honestly maintained lifecycle registry; the compiler cannot establish the truth of compromised upstream attestations.
12) **Comparative evidence deferred.** Until the preregistered RQ5 study is executed, no claim is made that the method outperforms unaided manual translation; the deferral is cryptographically committed, and this paper's claims are confined to the method, artifacts, formal results, and C1/C2 measurements.
13) **Scope of the structural and determinism claims.** Orphan-control impossibility and the exclusion of scalar priority from gate derivation are compiler guarantees over Φ-produced bundles (a signed OTHER_MANDATORY judgment may still be score-motivated; the guarantee is that it is explicit and attributable, not that it cannot occur); a deployment inherits them only under the runtime acceptance condition of Section VI-F. Determinism is measured inside a pinned reproducible container — repeated runs, key and row reordering, locale and time-zone variation — and reproduced on 8 independently installed environments spanning three operating systems, two architecture families, and two CPython minor versions; TD = 1.000 across those environments does not establish platform independence beyond them, and environment independence in general remains a design property of the semantics rather than a measured result.
14) **Cumulative influence partially exercised.** The disposition rule of Section VII-I is stated, not proved; Case C exercises it as an input-provenance field with human-response records (C-03, C-08, C-13) but does not evaluate whether that treatment is sufficient. Whether cumulative influence warrants a distinct consequence class remains unresolved, and the completeness of a declared input set is an evidence-production property this method does not warrant.

XIV. Relation to Prior Work

The L-DREA paper (doi 10.1109/ACCESS.2026.3719838) [15] contributes the enforcement authority and evidence layer; the theory paper (DOI 10.5281/zenodo.20369438) [16] the concurrence foundation and aggregator-class separation proof; the published patent application (US 2026/0127298 A1) [17] the enforcement mechanism claims. The present paper contributes the upstream refinement and compilation method, the gate-coverage formalization with its representability theorem and three propositions, the temporally valid traceability model, and the cyber-physical schema extensions; it consumes the L-DREA receipt interface (extended by the requirement-reference conformance condition of Section II) and repeats none of the enforcement architecture. The three documents answer three different questions — this paper: what should the executable controls be, where did they come from, and how do they change when governance changes; [16]: given mandatory predicates, what constitutes valid concurrence; [15]: at execution time, how are those predicates enforced so that an unauthorized action does not externalize — and do not re-present one another's theorems, experiments, tables, or figures as new contributions (this paper consumes and governance-wraps the downstream invariants; it does not re-derive them): [16] proves the aggregation primitive and its falsification surfaces; [15] specifies and evaluates the enforcement architecture that applies it at the externalization boundary; this paper specifies and evaluates the compilation layer that produces the predicate vector the other two take as given. The primary handoff object between the papers is the executable predicate bundle; the requirement-reference field on each decision receipt is its evidence-side counterpart. [15] begins at the authorization trace — transaction context, should-permit/should-deny ground truth, executable predicates, deterministic authority, PERMIT/DENY, evidence commit, externalization or none — and this paper explains where the predicates it evaluates come from: policy or regulatory requirement, risk register, control requirement, Ψ_K, structured control, Φ, executable predicate. The published downstream claims are unchanged by the upstream extension.

XV. Claim Boundary

(Format follows the normative claimed / not-claimed table established in [15].) The boundary is stated in Table VIII.

XVI. Conclusion

Governance frameworks end with an approved assessment; runtime authorities begin with an approved predicate set. This paper makes the step between them governed, total, deterministic, and traceable — in that order. Governed, because semantic refinement is a signed human act — a judgment record over an approved catalog — never a heuristic hidden inside a compiler, and never misdescribed as a function of the assessment alone. Total, because every risk receives exactly one explicit disposition, so no identified remainder can be silently accepted and non-runtime risks are honestly routed rather than force-translated — whether a remainder was correctly identified remains the empirical question the silent-narrowing rate measures. Deterministic, because after approval, compilation to a canonically serialized, signed, immutable bundle is designed to exclude every environment-dependent source of nondeterminism, is measured deterministic under the declared reproducible environment, and is adversarially testable, with lifecycle state carried in a separately signed registry rather than mutated in place. Traceable, because the authorized-origin chain — requirement to disposition to specification to predicate to receipt — is temporally valid and mechanically queryable, with authorization preceding evidence commit preceding actuation exactly as the consuming enforcement pipeline requires, turning "which requirement does this control serve, and was it in force when this decision was made?" from an audit reconstruction into a database join. And when an approved authorization assignment is not score-representable, scalar priority cannot determine gate membership — not as a preference but as a theorem: no monotone score threshold reproduces a gate assignment containing an inversion, no function of the scalar score expresses a collision with divergent gates, and the approved class-based assignment remains invariant under re-rating, under the stated hypotheses, through the improvement of the very controls it governs. Where the system acts on the world, the gate must know its place: it admits transitions and refuses trajectories, it commits before actuation and re-authorizes after a safety intervention, it binds a permit to the situation the authorizer saw and not only to the configuration the assessment covered, it names every party who shares or hands over authority, and it leaves the safety layer's vocabulary — and its loop — alone. The invariant that survives the change of domain is the one the enforcement layer already states: no valid permit, no external effect — whether the effect is a payment leaving an account or an instrument entering tissue. The immediate future work is fixed by the evaluation plan itself: compile and measure the Case C design case, execute the already hash-committed comparative study — then hand the compiled-bundle interface to the shadow-mode deployment and the cyber-physical conformance harness that a subsequent paper will require.

Acknowledgment

The cumulative-decision-influence case of Section VII-I was sharpened in discussion within the NIST AI RMF Trustworthy AI in Critical Infrastructure Profile community of interest. The views expressed are the author's own and do not represent NIST or any participant in that discussion. In accordance with IEEE policy on AI-generated text, the author discloses that large-language-model assistants (including Anthropic's Claude) were used for drafting and editing prose, consistency checking across the manuscript source, and bibliographic verification; all scientific judgments, formal results, artifacts, validation, and claims were reviewed by the author, who remains solely responsible for them.

Data and Code Availability

The GC-IR schema (v1.1), reference implementations of Ψ_K tooling and Φ, the Control Derivation Catalog, both C^* profiles, the Case A and Case B artifact sets (Steps 1–5 inputs, judgment records, dispositions, specifications, compiled bundles, lifecycle-registry, revocation, standing-permit, delegation, and safety-event fixtures), the Case C design specification (Steps 1–5 inputs, judgment record, dispositions, and specifications; no compiled bundle), coverage and temporal-validity queries, the adjudication instrument, and the preregistration commitment are released with this article as hash-pinned tagged release v1.0.3 (scientific freeze preregister-tier0-v2.2, commit c44f25d6fdb67e0bc4ac73a6217125dec8da1c0e) per Appendix C, which prints the release tag, freeze commit, manifest root hash, container digest, bundle payload hashes, and the DOI of the deposited tarball. The held-out expert-study register is withheld until study close, then released with the study data.

Disclosures

The author is the inventor on U.S. patent application publication US 2026/0127298 A1 [17] and the author of [15] and [16], and is the Principal Advisor of Gillian Holdings Incorporated, which holds commercial interests in enforcement-layer implementations of the cited architecture. Consistent with Section XII-C, the author does not participate in adjudication of the reference standard, and no reviewer with an authorship, ownership, or commercial interest in the method evaluates it.

Funding

This work received no external funding.

Appendix A
GC-IR Normative Schema Requirements (v1.1; Hash-Pinned at the Artifact-Repository Release)

The artifact schema uses separate record types: Record = CompiledPredicate **oneOf** NonRuntimeDisposition **oneOf** AcceptedRiskDisposition **oneOf** UnresolvedDisposition; registry REG = BundleLifecycle $\oplus$ ActorScopeRevocation; auxiliary = StandingPermit $\oplus$ SafetyEvent $\oplus$ HumanResponse (auxiliary records are signed, referenced from the bundle, and never inside the hashed payload).

TABLE VIII
CLAIM BOUNDARY.

Claimed	Not claimed
Silent narrowing defined over an adjudicated semantic reference, with total disposition as the artifact that makes it detectable and a preregistered instrument for its first measurement	That the method makes $SN_i = 0$ by syntax, or that the silent-narrowing rate has been measured in this paper (deferred to the committed study)
Governed refinement from assessed risk to an approved control specification, single-valued given the signed judgment record J	Automatic semantic interpretation of unconstrained risk prose; that Ψ_K is a function of the assessment alone
Deterministic compilation after event type, evidence semantics, and control meaning are approved, measured deterministic under the declared reproducible environment and reproduced on 8 tested host environments	That Φ determines what a legal or governance requirement ought to mean; environment independence beyond the tested environments as a measured result
Total disposition of every register row	That every risk should become a runtime predicate
Consequence-class gate coverage over a domain-parameterized profile, with the gate representability theorem (exact conditions for score- and threshold-representability, descriptor sufficiency), its monotone-threshold and score-collision propositions, and divergence from declared heat-map comparators measured on the financial artifact and deterministically recomputed on the cyber-physical register	That every possible risk-scoring method is compensatory
Gate stability under rating changes when action model and consequence class remain unchanged	That controls can never justify formal consequence reclassification
Origin-closed, temporally valid traceability from requirement to receipt, over an immutable bundle, signed lifecycle registry, and actor-scope revocations	That empty traceability queries prove substantive compliance
Fail-closed unknown- and evaluation-timeout handling required for mandatory gates, concurrence-window expiry as DENY, and explicit enforcement phases, propagated into the governance record; DENY and HOLD as distinct non-actuating outcomes	Elimination of Step 7 validation or human review
Transition admissibility (discrete, latency-bounded, evidence-bounded) as a compile-time check, with control-loop-rate hazards routed to a declared runtime safety layer	That the authorization gate mediates, supervises, or is faster than any control loop; trajectory or motion safety
Safety-layer interventions logged with standing origin and correlated to authorization state; interrupted transitions re-authorized via state binding	Correctness, coverage, or certification of the safety layer itself
World-state binding of permits with boundary re-validation; N-of-M concurrence, class-scoped standing permits, delegation by narrowing, actor-scope revocation, and human-response records as schema elements	That these elements are sufficient for any particular clinical, regulatory, or device-approval purpose
Orphan-control impossibility within Φ-produced bundles, extending to deployments under the stated runtime acceptance condition	Orphan-control impossibility for policy injected through channels outside that condition
An explicit upstream chain for the predicate family evaluated at volume in [15]	Any change to, or re-statement as new, the downstream results of [15]; detection capability
A preregistered, publicly hash-committed protocol for the comparative expert study	Comparative superiority over unaided manual practice (deferred to that study)
That Φ derives mandatory-gate membership from consequence class and declared authority and never from the scalar rating, so that any score-influenced OTHER_MANDATORY status exists only as an explicit signed judgment	That scalar priority can play no role in a governance authority's decision to declare a gate, or that such a declaration cannot occur
Independent meta-enforcement of evaluation deadlines and commit-before-actuate ordering, with INV-LATENCY and INV-EVIDENCE-COMMIT carried as marked invariant records outside the PERMIT aggregate	That an evaluator can reliably detect its own failure to complete, or that a receipt can serve as a pre-authorization predicate before the decision creating it exists
C1/C2 evidence with declared uncertainty, exposure denominators, and upper bounds	Independent certification (C3/C4), production effectiveness, clinical efficacy, product conformance, or real-world harm elimination

A non-runtime, accepted, or unresolved record shall not be required to contain subject, action, resource, evaluation basis, or gate type. A **CompiledPredicate** requires: predicate ID and ACS ID; authorized origin type and origin ID; risk and obligation references; subject, action, resource, and destination; action-parameter schema; at least one context condition; evidence producer and evidence schema; evaluation basis; gate type, gate source, and mandatory role; enforcement phase; on-fail, on-unknown, and on-evaluation-timeout, and concurrence-expiry response where a concurrence policy is present; evaluation latency bound with threshold contract; state binding where the action is C^*-classified; concurrence policy or standing-permit reference where the evaluation basis is `human_attestation` and the profile declares shared or pre-issued authority; delegation where the action is `DELEGATE`; escalation; evidence requirements; and version and validity bindings (`effective_from` only — lifecycle state is external per Section VI-F). A **NonRuntimeDisposition** requires: risk reference, reason code, routed-to control family $\in$ {`architectural`, `contractual`, `human_process`, `runtime_safety`}, control reference, owner, and approval record; `runtime_safety` additionally requires `safety_layer_id` $\in$ `S.architecture.safety_layers` and `standing_authorization_ref` = INV-SAFETY-RECOVERY. An **AcceptedRiskDisposition** requires: risk reference, scope, rationale, acceptor identity (resolving to `M.accountable_exec`), approval time, and expiry. An **UnresolvedDisposition** requires: risk reference, reason code, and the authority scope blocked from release.

Normative cross-field constraints (enforced by Φ, step 1 and bundle-level checks):

1) `mandatory` implies `on_fail = DENY`.
2) `mandatory` implies every required condition has `on_unknown = HOLD` and the specification has `on_evaluation_timeout = HOLD`; where a concurrence policy is present, `concurrence_expiry_response = DENY`.
3) Numeric or temporal operators require a threshold contract and unit.
4) `weighted` requires weight, normalized deficit function, aggregation group, and group threshold; the group re-

solves to a single group-level deficit at the enforcement boundary.

5) Every action tuple resolves to `S.authority_matrix`.
6) Every risk-derived predicate resolves to an ACS and risk.
7) Every compiler invariant resolves to an approved invariant ID in INV.
8) Every obligation reference resolves to O; an unresolved reference raises warning code WC-01 (not a disposition).
9) No expired approval may enter a release-admissible bundle.
10) Assessor identity may not equal acceptor identity (Section III, Step 1).
11) An accepted-risk disposition's acceptor resolves to `M.accountable_exec` and its expiry does not exceed the assessment validity interval; no release-admissible bundle contains an acceptance already past expiry at compile time.
12) Every risk with $C^*(c_i) = 1$ has at least one mandatory gate — or mandatory gate set — covering each of its authorized hazardous action paths (coverage requirement CV, Section VII-A).
13) The bundle payload contains no timestamp, nonce, or mutable lifecycle field; retirement, supersession, and revocation resolve only through signed lifecycle-registry records (Section VI-F).
14) **Runtime acceptance condition (deployment conformance).** A conformant consuming runtime loads only bundles verifying as signed Φ outputs — signature, payload hash, and lifecycle-registry state — accepts no policy content through any other channel, and emits decision receipts carrying the requirement reference (`gcir_id`, `acs_id`, and origin) of every evaluated predicate, the governance-layer outcome $\in$ {PERMIT, HOLD, DENY}, the state digest at authorization and at the boundary, and the actor and action tuple evaluated. This is the condition under which the compiler's orphan-control guarantee extends to the deployment, and the requirement-reference field is the interface extension of Section II.
15) **Human-response record.** A conformant consuming runtime emits a HumanResponse (`permit_id`, `response`, `responder`, `response_time`, `signature`) for every advisory predicate result and every `PRESENT_GUIDANCE` permit, within the declared response window.
16) **Transition admissibility.** Every action tuple in `S.authority_matrix` satisfies AD-1..AD-3; every specification declares `evaluation_latency_bound` with a threshold contract; a specification over an inadmissible tuple is rejected and the risk carries RC-06.
17) **State binding.** Every permit over a C^*-classified transition carries a `state_binding` digest; where a binding is declared, the digest is recomputed at the execution boundary, mismatch is HOLD, and the permit is not reusable; where no binding is declared, StateValid (Section VIII) holds vacuously and no digest is carried.
18) **Concurrence.** `concurrence_policy` is all-or-nothing: `n_required` attestations from distinct actors satisfying `role_constraints` within `window`, else DENY on window expiry (`concurrence_expiry_response`); an evaluator that does not finish inside its latency bound is HOLD (`on_evaluation_timeout`), never DENY; partial concurrence never accrues and is never carried forward.
19) **Standing permits.** A StandingPermit is matched by transition type only, satisfies only the `human_attestation` basis of its matched types, has validity $\subseteq$ the assessment validity interval, and covers a C^*-classified transition only under the issuance policy the approved profile declares (both supplied profiles: issuance under a `concurrence_policy`; Section V, commitment 7).
20) **Delegation by narrowing.** A `DELEGATE` permit's child has scope $\subseteq$ parent.scope, validity $\subseteq$ parent.validity, action set $\subseteq$ parent action set, with at least one of the three strictly smaller (composite $\subsetneq$), and cites `parent_permit_id`; a child equal to its parent or not contained in it is rejected at issuance; revocation of a parent (bundle-level or actor-scope) voids its children transitively.
21) **Safety-event correlation.** Every intervention of a declared safety layer produces a signed SafetyEvent with origin INV-SAFETY-RECOVERY, a valid `safety_layer_id`, and correlation to the permit or bundle in force; SafetyEvent records are excluded from the ordering predicate over constraint-14 receipts and tested by audit query 8; the obligation is `POST_EVENT_AUDIT` — a SafetyEvent missing after the recording window Δ_{se} declared as `S.architecture.safety_layers[i].safety_event_w` for the intervening layer places HOLD on the next ordinary authorization in the affected scope and never conditions the intervention.
22) **Enforcement phase.** Every CompiledPredicate declares `enforcement_phase`; only `PRE_AUTHORIZATION` and `BOUNDARY_REVALIDATION` predicates enter the PERMIT aggregate; `POST_AUTH_PRE_ACTUATION` predicates are evaluated at interlock release and block release on failure; `POST_EVENT_AUDIT` predicates are evaluated after the event and resolve to HOLD on the next ordinary authorization in scope.
23) **Judgment reference.** Every record of D and Δ carries `judgment_ref` = (`J.id`, `J.version`, `J.hash`); Φ verifies the reference and includes `J.hash` in the canonical payload.
24) **Meta-enforcement and standing-origin separation.** INV-LATENCY and INV-EVIDENCE-COMMIT carry `meta_enforcement` = true and shall not enter the non-compensatory PERMIT aggregate; INV-LATENCY declares an independent deadline watchdog as its discharge mechanism and INV-EVIDENCE-COMMIT the commit-before-actuate interlock. INV-SAFETY-RECOVERY carries `enforcement_phase`

= POST_EVENT_AUDIT and standing origin; it shall not enter the aggregate. Every such record resolves to an approved invariant ID in INV (Eq. (1)).

25) **Control-loop declarations.** S.architecture.control_loops lists (control_loop_ref, control_loop_period, excluded_action_family) per excluded family; every RC-06 disposition cites a control_loop_ref.

The machine-readable JSON Schema (draft 2020-12) implementing these requirements ships in the repository as the frozen v1.1 schema.

Example record. The Case A R-05 compiled predicate, as serialized (fields abbreviated):

```
{
 "record_type": "CompiledPredicate",
 "gcir_id": "GCIR-0007",
 "acs_id": "ACS-0005-01",
 "origin_type": "risk_derived",
 "requirement_ref": {"risk_id": "R-05",
   "obligation_ids": ["CIRO-3608-DISC", "SEC-BAR-INT"]},
 "subject": "research_agent",
 "action": "publish_report",
 "resource": "research_report",
 "destination": "internal_distribution",
 "context_conditions": [
   {"attribute": "restricted_list_match", "operator": "==",
    "value": false, "on_unknown": "HOLD",
    "evidence_producer": "entity_resolution_service_v3"}
 ],
 "evaluation_basis": "deterministic_lookup",
 "enforcement_phase": "PRE_AUTHORIZATION",
 "evaluation_latency_bound": {"value": 250, "unit": "ms",
   "threshold_contract_ref": "TC-LAT-01"},
 "on_evaluation_timeout": "HOLD",
 "gate_type": "mandatory",
 "gate_source": "C_STAR",
 "on_fail": "DENY",
 "escalation": {"route": "supervisory_analyst", "sla_hours":
     4},
 "evidence_requirement": ["source_hashes",
   "disclosure_checklist_id", "reviewer_id"],
 "version_binding_ref": "M.version_binding",
 "validity": {"effective_from": "..."}
}
```

Audit queries. Over a conformant evidence store, the following queries must return empty result sets (Section VIII):

1) obligations without an approved disposition;
2) risks without exactly one disposition;
3) predicates without a risk or compiler-invariant origin;
4) receipts whose bundle was not valid at decision time (payload hash, registry state, actor-scope state, or state digest);
5) AuthorizedActuation records lacking a temporally prior committed receipt whose authorization time is not later than its commit time;
6) lifecycle-registry records lacking a valid signing authority;
7) AuthorizedActuation records whose actor–scope was revoked with effective_time $\leq$ authorization_time, or whose permit descends from a revoked parent;
8) SafetyEvent records lacking standing origin, lacking a valid safety-layer identifier, or lacking correlation to the authorization state in force at effective_time;
9) child permits whose scope or validity is not strictly contained in the parent's, or whose parent is absent;
10) advisory permits lacking a HumanResponse record within the declared response window.

APPENDIX B
Φ PSEUDOCODE (REFERENCE SEMANTICS)

```
function Phi(M, S, O, R, A, K, C_star, J, D, Delta, INV):
    validate_signatures_and_versions(M, S, O, R, A, K,
     C_star, J, D, Delta, INV)
        # C_star is a signed governance object versioned
     with M; its version is
        # bound into the payload so that a profile change
     changes the bundle hash
    require(all records of D, Delta cite judgment_ref == (J.
     id, J.version, J.hash))
        # D and Delta are J's products (Sec. IV-A); J enters
      Phi only by this reference
    validate_all_schemas()
    validate_exactly_one_disposition_per_risk(R, Delta)
    if exists r in R where Delta[r].status == unresolved:
        halt(RELEASE_INADMISSIBLE, r.risk_id)

    P <- {}; G <- {}; E <- {}

    for r in canonical_order(R):
        disposition <- Delta[r]
        if disposition.status in {nonruntime, accepted}:
            if disposition.routed_to == runtime_safety:
                require(disposition.safety_layer_id in S.
     architecture.safety_layers)
                require(disposition.
     standing_authorization_ref == INV_SAFETY_RECOVERY)
            continue                          # recorded in
     Delta; traceable via coverage matrix
        for acs in canonical_order(disposition.acs_records):
            template <- exact_lookup(K, acs.event_type, acs.
     template_id)
            if template == BOTTOM:
                halt(UNRESOLVED_TEMPLATE, acs.acs_id)   #
     heuristic matching prohibited
            validate_evidence_semantics(acs, template)
            validate_authority_tuple(acs.subject, acs.action
     , acs.resource,
                                     acs.destination, acs.
     parameter_schema,
                                     S.authority_matrix)
            validate_admissibility(acs.action_tuple, S.
     architecture.control_loops,
                                   acs.
     evaluation_latency_bound)   # AD-1 (semantic ^ timing),
      AD-2, AD-3; else RC-06(control_loop_ref)
            require(acs.enforcement_phase in {
     PRE_AUTHORIZATION, BOUNDARY_REVALIDATION,

     POST_AUTH_PRE_ACTUATION, POST_EVENT_AUDIT})
            if acs.action in C_star_actions(A, C_star):
                require(acs.state_binding != BOTTOM)
             # constraint 17
            if acs.evaluation_basis == human_attestation:
                validate_attestation_form(acs.
     concurrence_policy,
                                          acs.
     standing_permit_ref, S)  # constraints 18-19
            if acs.action == DELEGATE:
                require(acs.delegation != BOTTOM)
             # constraint 20
            gate_source <- classify_gate_source(acs, A,
     C_star)
            gate <- mandatory if (gate_source == C_STAR and
     acs.mandatory_role == decisive)
                                  or gate_source ==
     OTHER_MANDATORY
                    else approved_soft_gate(acs)
            # a C* risk may carry supporting advisory/
     weighted predicates;
                # coverage is asserted at bundle level (
     below), not per predicate
            if gate == mandatory:
                require(acs.on_fail == DENY)
                require(all_required_unknowns_hold(acs))
                require(acs.on_evaluation_timeout == HOLD)
                if acs.concurrence_policy != BOTTOM:
                    require(acs.concurrence_expiry_response
     == DENY)   # expiry != evaluator timeout
```

```
        recs <- instantiate_predicates(template, acs)
        require(len(recs) >= 1)      # every runtime ACS
  emits at least one record
        for rec in recs:
            rec.origin_type <- risk_derived
            rec.origin_id   <- r.risk_id
            rec.acs_id      <- acs.acs_id
            P <- P U {rec}
            G[rec.id] <- (gate, gate_source)
            E[rec.id] <- acs.escalation

for inv in canonical_order(INV):
    rec <- instantiate_invariant(inv, M, S)
        # e.g., INV-VERSION, INV-EVIDENCE-COMMIT, INV-
 AUTHORITY-CLOSURE,
        #       INV-STATE-BINDING, INV-LATENCY, INV-
 DELEGATION-NARROWING,
        #       INV-SAFETY-RECOVERY
    rec.enforcement_phase <- inv.phase
        # INV-VERSION / INV-AUTHORITY-CLOSURE / INV-
 LATENCY / INV-STATE-BINDING:
        #     PRE_AUTHORIZATION + BOUNDARY_REVALIDATION
        # INV-DELEGATION-NARROWING: PRE_AUTHORIZATION (
 issuance of child)
        # INV-EVIDENCE-COMMIT:  POST_AUTH_PRE_ACTUATION
        # INV-SAFETY-RECOVERY:  POST_EVENT_AUDIT
    rec.origin_type <- compiler_invariant
    rec.origin_id   <- inv.invariant_id
    rec.on_unknown             <- HOLD     # mandatory:
 required by constraint 2
    rec.on_evaluation_timeout <- HOLD     # mandatory:
 required by constraint 2
    rec.on_fail                <- inv.failure_response
        # declared per invariant in INV: INV-AUTHORITY-
 CLOSURE and
        # INV-DELEGATION-NARROWING -> DENY (evaluated
 prohibition);
        # INV-STATE-BINDING and INV-VERSION -> HOLD (
 authority cannot
        # presently be established; re-evaluation
 required); INV-LATENCY
        # -> HOLD via the watchdog; INV-EVIDENCE-COMMIT
 -> blocked release
    rec.meta_enforcement      <- (inv in {INV-LATENCY,
 INV-EVIDENCE-COMMIT})
        # discharged by a mechanism independent of the
 evaluator (Sec. VI-B)
    P <- P U {rec}
    G[rec.id] <- (mandatory, OTHER_MANDATORY)
    E[rec.id] <- inv.escalation

assert exactly_one_disposition_per_risk(R, Delta)
assert all_predicates_have_authorized_origin(P, R, INV)
assert all_action_tuples_are_authorized(P, S)
assert all_action_tuples_admissible(P, S)
assert no_mandatory_unknown_warns(P)
assert only_preauth_phases_in_permit_aggregate(P)
assert all_c_star_permits_state_bound(P, A, C_star)
assert safety_layers_declared_for_rc06(Delta, S)
assert c_star_coverage_satisfied(R, A, C_star, P, G)
    # CV: each C* risk's authorized hazardous action
 paths carry >= 1 mandatory gate

CM <- coverage_matrix(O, R, Delta, D, INV, P)
payload <- RFC8785_canonicalize(M.version_binding, K.
 version, C_star.version,
                                J.hash, Delta, P, G, E,
 CM)
    # canonical key ordering; deterministic array order;
  explicit numerics/units;
    # no timestamp, nonce, or mutable lifecycle field
 inside payload ---
    # retirement/supersession/revocation live in the
 separately signed registry REG
h <- SHA256(payload)
envelope <- sign_hash(h, signer_key_id =
 governance_forum_key_id,
                      signing_time = trusted_time())
    # signing time lives in the envelope, outside the
 hashed payload
return Bundle(payload, h, envelope)          # immutable
  after signing
```

Determinism argument: every source of nondeterminism is closed — iteration order (canonical sort), serialization (RFC 8785), encoding (fixed UTF-8), time (excluded from payload), lifecycle state (external registry), actor authority (external registry), safety-layer state (external SafetyEvent records), matching (exact lookup only), randomness (none). TD (Section XII) is therefore a *check* of implementation fidelity to this semantics, not a hope; the adversarial suite (Section XII-F) probes the closure.

APPENDIX C
CASE ARTIFACTS AND REPOSITORY

Artifact release. Artifacts are released as a hash-pinned tagged release under the author's GitHub organization, following the precedent of the repository of record named in [15], Appendix C — github.com/AGLakhowal/Gamma-Permit-Package — as a dedicated `gc-ir/` release tree within that repository — upstream of the artifact's L3 boundary layer, whose Γ-state computation evaluates the bundle this tree produces ([16], Table II) — MIT license for code and schema, per that precedent. The public commitment is the following fixed block:

- artifact release tag: `v1.0.3`; scientific freeze tag: `preregister-tier0-v2.2`;
- freeze commit (equal to the execution commit of every reported result): `c44f25d6fdb67e0bc4ac73a6217125dec8da1c0e`;
- `MANIFEST.sha256` root hash: `[MANIFEST.sha256 root hash]`;
- pinned container digest: `[container digest]`;
- Case A compiled bundle payload SHA-256: `f5cbc3a8016159d2074005fa021bf76ca04fb7aed1408f5ec845418460a43536`;
- Case B compiled bundle payload SHA-256: `2850155a2ee7d4c01db4748a891891bae27c48f4c9c2c7eccd30beb7d1aa33ce`;
- externally timestamped deposit of the tagged tarball: DOI `10.5281/zenodo.XXXXXXXX`.

A tag alone is movable; the deposit supplies the timestamp that Section XII-G relies on, and the manifest root hash binds every released file to it. The release contains: the GC-IR schema (v1.1); Ψ_K tooling and the Φ reference implementation; the Control Derivation Catalog; both C^* profiles; Steps 1–5 serialized inputs, signed judgment records, dispositions, and Approved Control Specifications for Cases A, B, and C; compiled bundles for Cases A and B (Case C is released as a design specification without a compiled bundle) and lifecycle-registry, revocation, standing-permit, delegation, and safety-event fixtures; coverage and temporal-validity queries; the adjudication instrument; the preregistration commitment (the complete RQ5 protocol: design, recorded fields, reliability statistics, Bayesian model, priors, convergence criteria, stopping rule); the frozen failure-injection matrices (thirteen Case B scenarios, nineteen Case C injections); and the cumulative-decision-influence note referenced from Section VII-I. The release also carries `tools/freeze_check.py`, which recomputes every measured figure reported in this paper (gate divergence, the three non-runtime residual ratings, the determinism run design, and the Monte Carlo maximum) from the released

dispositions, compiled bundles, run manifest, and Monte Carlo results, and `tools/montecarlo.py`, which regenerates the rating-robustness analysis from the released frozen rating distributions and seed. The held-out expert-study register is withheld until study close. The cyber-physical conformance harness of Section XI (synthetic and ROS 2 adapters, safety-layer adapter, scenario files for the E1–E19 matrix, and the enforcement-layer experiment scripts) is released separately with the follow-up paper that reports its results.

Hash manifest structure. One `MANIFEST.sha256` at release root listing, per file: path, SHA-256, and role $\in$ {schema, catalog, c_star_profiles, psi_tooling, phi_impl, judgment_record_a, judgment_record_b, judgment_record_c, case_a_input, case_b_input, case_c_input, dispositions_a, dispositions_b, dispositions_c, acs_a, acs_b, acs_c, compiled_bundle_a, compiled_bundle_b, lifecycle_registry, revocation_fixtures, standing_permit_fixtures, delegation_fixtures, safety_event_fixtures, queries, adjudication_instrument, prereg_commitment}. The release tag, manifest root hash, and deposit DOI together constitute the public preregistration commitment referenced in Section XII-G; the release also carries the run manifest of the 62 determinism executions and the cross-platform reproduction records for the 8 environments reported in Section XII-F.

APPENDIX D
CASE DETAIL: ROW NOTES, FAILURE-INJECTION MATRICES, AND CONFORMANCE HARNESS

This appendix carries the case material that supports Sections X and XI but is not needed to follow the argument of the main text. Nothing here is measured evidence; the Case B rows are inputs to the adversarial suite of Section XII-F, and the Case C rows are design-time injections over a register that is not compiled in this paper.

A. Case B: the full thirteen-predicate correspondence

Table IX crosswalks each of the thirteen check-level predicates that [15] evaluated at volume to the Case B specifications, compiler invariants, lifecycle structures, and registry state from which its authority derives, together with its failure outcome under the DENY/HOLD semantics of Section V. The correspondence is many-to-many — one specification may inform several checks (ACS-B01-01 informs authentication, token validity, scope, and state binding) and one check may draw authority from more than one origin (P3 from ACS-B02-01 and ACS-B02-02; P13 from B-06 and INV-EVIDENCE-COMMIT) — and it is not a GC-IR record-cardinality assertion: the compiled Case B bundle carries 9 records (Section X), and the thirteen downstream checks are its runtime realization, not its record count. The check-to-record mapping is recorded in the release manifest under role `case_b_input`.

TABLE IX
CASE B: ORIGIN OF THE THIRTEEN RUNTIME PREDICATES OF [15], RESOLVED TO CASE B SPECIFICATIONS AND COMPILER INVARIANTS. [15] DESCRIBES THE THIRTEEN AT THE CHECK LEVEL — SIGNATURE, SCOPE, EXPIRY, SINGLE-USE, REVOCATION, NON-COMPENSATORY NODE RISK, PERSISTENT CLASS-LEVEL VETO, CONTEXT/ISB, WATCHDOG (SECTIONS IX-A, IX-I) — AND NAMES THEM ONLY IN ITS RELEASED ARTIFACT; THE ULB TRACE NATIVELY FALSIFIES FOUR OF THEM AND THE SYNTHETIC SUITE THE REST (SECTION IX-H). THE NAMES BELOW ARE CASE B'S DOMAIN-LEVEL RENDERING; THE MAPPING TO THE ARTIFACT'S PREDICATE IDENTIFIERS IS RECORDED IN THE RELEASE MANIFEST. THE ESSENTIAL PROPERTY IS THAT ALL MANDATORY MEMBERS ARE NON-COMPENSATORY.

Predicate	Origin	Failure outcome
P1 account_active	B-01 (state_binding attribute)	HOLD (unknown) / DENY (frozen)
P2 customer_authenticated	B-01 ACS-B01-01	HOLD
P3 transaction_within_authority	B-02 ACS-B02-01 (ceiling) / ACS-B02-02 (escalation)	DENY above absolute ceiling; above ordinary limit: HOLD while approval pending or unavailable, DENY if denied or window expired
P4 beneficiary_permitted	B-04	DENY (restricted) / HOLD (unknown)
P5 sanctions_clear	B-04	DENY / HOLD (service unavailable)
P6 transaction_limit_valid	B-02, B-03	DENY
P7 state_digest_match	INV-STATE-BINDING (digest at the boundary equals digest at authorization; snapshot freshness is a threshold contract on the bound attributes)	HOLD
P8 required_approval_present	B-01 ACS-B01-03	HOLD while pending or unavailable; DENY if denied or concurrence window expired; evaluator timeout HOLD
P9 token_valid	B-01 ACS-B01-01	DENY
P10 token_not_revoked	ActorScopeRevocation (query 7)	DENY
P11 policy_version_valid	INV-VERSION	HOLD
P12 risk_prohibition_absent	B-01 ACS-B01-02	DENY
P13 decision_lineage_valid	B-06 (risk-derived) / INV-EVIDENCE-COMMIT (compiler-invariant, POST_AUTH_PRE_ACTUATION)	lineage not established: interlock release blocked, no actuation; two authorized origins, one mechanism

B. Case B failure-injection rows (summary)

The adversarial suite carries thirteen Case B scenarios, each of which must produce no externalization, in five categories: (i) prohibition present despite favourable evidence elsewhere (restricted beneficiary with a low fraud score; invalid actor authority with a valid token; amount above the actor's ceiling) — DENY; (ii) required condition unresolved (sanctions service unavailable; stale account or token snapshot; policy-version mismatch) — HOLD, service failure never read as clearance; (iii) state changed between proposal and boundary (account frozen, token revoked, beneficiary altered) — ActorScopeRevocation and INV-STATE-BINDING at the boundary, never execution under stale authority; (iv) integrity (single-use permit replayed; conflicting mandatory policies without approved

precedence) — DENY and HOLD respectively, both controls recorded; (v) cognition and authority separated (a correct model with an unauthorized releasing agent; an incorrect model with a deterministic screening predicate) — DENY on the authority or screening predicate alone, plus the all-concur PERMIT control. The complete frozen matrix, one row per scenario with its injected condition, expected outcome, and the requirement each violates, is released with the artifact.

C. Case C obligation matrix and runtime context

Runtime context consumed by the predicates: surgeon identity and credentials, active console, control owner and control token, procedure class and step, instrument identity and capability state, requested action, autonomy mode, robot operating state, context timestamp, policy version. Obligations (Step 3): ISO 14971 risk-control and residual-risk requirements (`standards`); IEC 62304 software safety classification of the assistance module (`standards`); IEC 80601-2-77 particular requirements for basic safety and essential performance of RASE (`requirement_class` = `standards` in the profile as supplied, promoted to statutory only where the deploying jurisdiction's regulator incorporates the standard by reference — a Step 3 applicability decision recorded in O, not assumed here); manufacturer IFU contraindications (contractual — an obligation on the deploying institution under the device's conditions of use); institutional privileging and credentialing (internal); consent scope for intraoperative video and telemetry (statutory privacy plus internal).

D. Case C row notes (continued)

C-02 note (world-state binding). A permit to deliver energy that was valid when the instrument was at step 6 with port configuration α must not remain valid at step 7 with configuration β merely because the model version, prompt, and policy are unchanged. `state_binding` closes the gap that `version_binding` leaves: the permit is digested over the declared state tuple, recomputed at the execution boundary, and mismatched $\rightarrow$ HOLD. Case B's transaction-hash binding is the special case in which the state tuple is the payload; the surgical case is why the field had to be generalized.

C-05 note (revocation is not retirement). "The surgeon withdraws authority" retires nothing: the bundle is still in force for every other actor and scope. An ActorScopeRevocation record withdraws one actor's authority over one scope from one effective time, voids that actor's standing and delegated permits in the scope transitively, and is tested by audit query 7. Bundle-level revocation would have been the wrong instrument — too wide, and it would have retired the safety-correlation and data-plane predicates along with the surgeon's. Authority that changes between proposal and actuation is caught at the boundary, not at proposal time.

C-08 / C-13 note (advisory outputs, lineage, and Section VII-I exercised). Guidance is not actuation, but a sequence of accepted guidance is the operational picture on which the next actuation is authorized. C-08 gates the *presentation* — contraindicated or stale guidance is suppressed by a structural check — and the HumanResponse record on each presented item shows whether it was followed or deviated from. C-13 is the tier-1 defect: a snapshot that is stale, belongs to the wrong patient, or lacks lineage. Neither is confined to its tier: C-03's `input_provenance` names the guidance permits and the snapshot lineage on which the autonomous proposal rests, so a provenance defect at the data tier yields HOLD at the actuation tier whose decision materially depended on it, even though the later controller functions correctly and the trajectory is admissible. Re-auditing the accumulated picture at authorization time remains intractable; checking that the declared inputs exist, were authorized, and carry responses is a join.

C-09 note (the data plane inside the room). Intraoperative video and telemetry are the raw material of the assistance module's next model, and their export is a transition with an authority matrix row of its own. The generalized class `protected_data_disclosure_outside_consent_scope` is the finance information-barrier breach with the barrier replaced by a consent record: the same structural check (signed artifact covers the tuple) against a different artifact.

C-11 / C-12 note (standing permits and delegation). Pre-authorized routine steps are a StandingPermit matched by transition type; the standing permit satisfies attestation and nothing else, so state, version, and context are still evaluated per step. C-11 monitors the attestation channel that remains for consequential steps, as R-12 monitors reviewer dwell. C-12 is the delegation invariant: a surgeon may hand a bounded subtask to autonomy for a bounded interval, and the child permit's (scope, validity, actions) must be strictly contained ($\subsetneq$) in the parent's — contained on every component, smaller on at least one — rejected at issuance otherwise, and voided with the parent on revocation.

C-14 / C-15 note (the two invariants a robotics reader will look for first). Neither row needs a register parent to exist: INV-LATENCY and INV-SAFETY-RECOVERY are emitted by Φ for every bundle with a declared safety layer. The rows do what R-11 does for INV-VERSION — attach ownership, rating, treatment, and escalation to an invariant — and their phases keep them honest. A governor that can be late must say what lateness means, and cannot be the one to say it: INV-LATENCY is a meta-enforcement invariant discharged by a deadline watchdog outside the evaluator (Section VI-B), and C-14 attaches ownership, the latency threshold contract, escalation, and evidence requirements to that watchdog; lateness means HOLD with no partial permit, at `PRE_AUTHORIZATION`. A safety layer that can fire must be visible in the evidence; INV-SAFETY-RECOVERY is `POST_EVENT_AUDIT`: it obliges a signed SafetyEvent within Δ_{se} of any intervention and, if the correlation is missing, places HOLD on the next ordinary authorization in the affected scope. C-15 therefore makes an unrecorded stop a control failure rather than an unremarkable event — and it does not, and could not, gate the stop itself.

E. Case C failure-injection matrix (summary)

The Case C matrix carries nineteen design-time injections (E1–E19) over the printed register, in six categories: au-

TABLE X
CAPABILITY TIERS RECORDED IN S.AUTONOMY_LEVEL FOR CASE C, THE GOVERNANCE QUESTION EACH TIER RAISES, AND THE REGISTER ROWS THAT EXERCISE IT. THE FIVE-TIER PROGRESSION FOLLOWS THE PUBLICLY DESCRIBED INDUSTRY CAPABILITY LADDER OF [44] AND IS USED HERE AS A PROFILE-TIME SCOPING DEVICE.

Tier	Governance question	Transition(s)	Rows
1 Data	Is this data valid, provenanced, current, and within consent scope for this use?	EXPORT_DATA; evidence-producing	C-09, C-13
2 Insight	Is this model authorized for this patient, procedure class, and intended use?	evidence-producing	C-10
3 Guidance	May this guidance be presented now, in this context — and was it followed?	PRESENT_GUIDANCE	C-08
4 Augmentation	May the system modify this physical action?	ACTIVATE_INSTRUMENT, ENERGY_ACTIVATE	C-01, C-02
5 Supervised autonomy	May the autonomous system execute this consequential transition, and who is answerable?	ENTER_AUTONOMOUS_SUBTASK, FIRE_STAPLER, TRANSFER_CONTROL, DELEGATE	C-03, C-04, C-05, C-12
— Safety layer	(not a governance question — standing authority)	none	C-06, C-07, C-15

thority and identity (unprivileged supervisor, expired credential, control-token mismatch, mode mismatch); state binding (procedure-step, instrument, port-configuration, and patient-position digest mismatches at the boundary); concurrence and delegation (window expiry with one attestation — DENY via concurrence_expiry_response; evaluator timeout — HOLD; child permit wider than its parent — rejected at issuance); revocation (actor-scope withdrawal mid-subtask; cascade to delegated children); safety-layer interaction (interlock stop under a valid permit — resumption HOLD; unrecorded intervention — HOLD on the next ordinary authorization, E19); and evidence (missing human-response record; lineage defect at the data tier blocking the actuation-tier permit through input_provenance). Each injection names the row, the invariant or specification exercised, and the expected outcome; none is executed in this paper (Case C is a design case), and the complete matrix is frozen in the artifact release.

F. Conformance harness (future work, not claimed)

Conformance harness (future work, not claimed). The planned environment for compiling and exercising the Case C bundle is a three-stage ladder: (1) a deterministic synthetic harness plus a kinematic research-robot simulator exposing the same ROS 2 state topics — joint state, Cartesian pose, operating and control mode, jaw/instrument state, teleoperation state, timestamps — as research hardware [40], with synthetic procedure, policy, authority, and provenance context assembled alongside; (2) dynamic surgical simulation where contact and force are represented; (3) research hardware with the same ROS-facing interface, which is research infrastructure and not validation of any clinical product. At each stage the bundle is loaded by an L-DREA-class authority and the metrics reported are the six primary metrics of [15], Section VIII-G — false-permit rate, false-denial rate, replay-determinism rate, revocation compliance, TOCTOU violation rate, and class-veto effectiveness — with its operational definition of unauthorized execution (eq. 9 there), its decision-path latency reporting (Table 19 there), and, added for this domain, the six-cell authority×safety outcome matrix of Table IV; none of these is measured in this paper. The experimental question at every stage is the same: did the authority completely mediate the consequential transition, evaluate the compiled predicates deterministically against current state, commit the receipt before any permitted actuation, fail closed whenever a valid permit could not be established, leave the safety layer's standing authority untouched, and produce replayable evidence?

APPENDIX E
CONTINUOUS RATING-SENSITIVITY BOUND (EXPLANATORY)

This bound is explanatory only; the empirical analysis of Section XII-E is discrete. For the continuous surrogate $s(L, I) = L \cdot I$, the multivariate mean-value theorem gives, for perturbation $\delta = (\delta_L, \delta_I)$:

$$|s(z+\delta)-s(z)| \le \sup_{\xi \in [z, z+\delta]} \|\nabla s(\xi)\|_2 \|\delta\|_2, \quad \nabla s(\xi) = (\xi_I, \xi_L),$$

hence $|s(z+\delta) - s(z)| \le \sup_\xi \sqrt{\xi_L^2 + \xi_I^2} \cdot \|\delta\|_2$. This bound explains why scalar gate membership can be unstable for risks close to a threshold: small rating changes may cross the authorization boundary even when the underlying consequence class is unchanged. Because enterprise ratings are ordinal rather than truly continuous, the bound is explanatory; the empirical robustness analysis uses discrete Monte Carlo sampling (Section XII-E).

REFERENCES

[1] NIST, "Artificial Intelligence Risk Management Framework (AI RMF 1.0)," NIST AI 100-1, Jan. 2023.

[2] ISO/IEC 42001:2023, *Information Technology — Artificial Intelligence — Management System*, ISO/IEC, Dec. 2023.

[3] ISO 31000:2018, *Risk Management — Guidelines*, ISO, Feb. 2018.

[4] OSFI, Guideline E-23, Model Risk Management, published Sep. 11, 2025 (effective May 1, 2027); OSFI Guideline B-10, Third-Party Risk Management (2023); Guideline B-13, Technology and Cyber Risk Management (2022); Guideline E-21, Operational Risk and Resilience (2024); OSFI, "Generative and Agentic Artificial Intelligence: Implications for Technology, Cyber Security, and Operational Resilience," Technology Risk Bulletin, Jul. 1, 2026 (a supervisory communication describing sound practices; not a regulatory expectation).

[5] Board of Governors of the Federal Reserve System / OCC, "Supervisory Guidance on Model Risk Management," SR 11-7 / OCC 2011-12, Apr. 4, 2011.

[6] CIRO, Investment Dealer and Partially Consolidated Rules, Rule 3600 (Research Reports), esp. ss. 3608–3622.

[7] Regulation (EU) 2024/1689 (EU AI Act), Art. 113; GPAI obligations applicable from Aug. 2, 2025; as amended by Regulation (EU) 2026/1744 (Digital Omnibus on AI), in force July 27, 2026, deferring application of the high-risk obligations to Dec. 2, 2027 (Annex III) and Aug. 2, 2028 (Annex I) and leaving the Art. 50 transparency obligations, incl. the Dec. 2, 2026 marking deadline, unchanged.

[8] OASIS, "eXtensible Access Control Markup Language (XACML) Version 3.0," OASIS Standard, Jan. 2013; Open Policy Agent, "Policy Language (Rego)," OPA documentation, openpolicyagent.org/docs (accessed Sep. 2026).

[9] Assurance Case Working Group, "GSN Community Standard Version 3," SCSC-141C, 2021.
[10] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," NIST SP 800-207, Aug. 2020.
[11] J. P. Anderson, "Computer Security Technology Planning Study," ESD-TR-73-51, USAF Electronic Systems Division, Oct. 1972.
[12] IEC 61511:2016, *Functional Safety — Safety Instrumented Systems for the Process Industry Sector* / ANSI/ISA-84.00.01.
[13] NIST, "Trustworthy AI in Critical Infrastructure Profile (TACIP)," Community of Interest discussion draft, version of Aug. 3, 2026. Unpublished, COI-distributed document; cited as a versioned discussion draft (date and SHA-256 recorded in the artifact repository), not as an established NIST publication. Practices 3 and 12; "Strategic Governance & Executive Translation" open design area.
[14] SEC, In the Matter of Knight Capital Americas LLC, Admin. Proc. File No. 3-15570, Oct. 16, 2013; Market Access Rule, 17 CFR §240.15c3-5.
[15] A. Gill-Lakhowal, "Deterministic Runtime Enforcement: The Execution Authority for Autonomous AI Agents," *IEEE Access*, vol. 14, pp. 119764–119790, 2026, doi: 10.1109/ACCESS.2026.3719838.
[16] A. Gill, "The Lakhowal Law of Concurrence: A Deterministic Authorization Primitive for Runtime AI Governance with Hardware-Rooted Closure and Paired Falsification Surfaces," Zenodo, 2026, doi: 10.5281/zenodo.20369438.
[17] A. Gill, "The Lakhowal reverse law: Deterministic runtime proof and federated AI control systems," U.S. Patent Application 19/383,841, Publication US 2026/0127298 A1, May 7, 2026.
[18] S. H. Jayasundara, N. A. G. Arachchilage, and G. Russello, "SoK: Access Control Policy Generation from High-Level Natural Language Requirements," *ACM Computing Surveys*, vol. 57, no. 4, art. no. 104, pp. 1–37, 2025 (online 23 Dec. 2024), doi: 10.1145/3706057.
[19] M. Narouei, H. Takabi, and R. Nielsen, "Automatic Extraction of Access Control Policies From Natural-Language Documents," *IEEE Transactions on Dependable and Secure Computing*, vol. 17, no. 3, pp. 506–517, 2020, doi: 10.1109/TDSC.2018.2818708.
[20] J. Mavračić, "Policy Cards: Machine-Readable Runtime Governance for Autonomous AI Agents," arXiv:2510.24383, 2025.
[21] R. Cilla Ugarte, M. Á. Patricio Guisado, A. Berlanga de Jesús, and J. M. Molina López, "Making AI Compliance Evidence Machine-Readable," arXiv:2604.13767, 2026.
[22] A. K. Sharma and J. M. Kunkel, "Ontological Knowledge Blocks: Executable Compliance and Profile-Based Validation for Trustworthy AI Systems," arXiv:2605.23297, 2026.
[23] U. Uchibeke, "Before the Tool Call: Deterministic Pre-Action Authorization for Autonomous AI Agents," arXiv:2603.20953, Mar. 2026.
[24] C. Koch, "From Governance Norms to Enforceable Controls: A Layered Translation Method for Runtime Guardrails in Agentic AI," arXiv:2604.05229, Apr. 2026.
[25] N. Palumbo, S. Choudhary, J. Choi, P. Chalasani, and S. Jha, "Policy Compiler for Secure Agentic Systems," arXiv:2602.16708v2, Feb. 2026.
[26] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents," in *Proc. IEEE/ACM 48th Int. Conf. Softw. Eng. (ICSE)*, Rio de Janeiro, Brazil, Apr. 2026; arXiv:2503.18666.
[27] G. V. Datla, A. Vurity, T. Dash, T. Ahmad, M. Adnan, and S. Rafi, "Executable Governance for AI: Translating Policies into Rules Using LLMs," in *AAAI-26 Workshop on AI Governance*, arXiv:2512.04408, Dec. 2025.
[28] P. Madatha, "A Deterministic Control Plane for LLM Coding Agents," arXiv:2606.26924, Jun. 2026.
[29] L. G. Iyer and R. S. Babu, "Capability Minimization as a Safety Primitive: Risk-Aware Causal Gating for Least-Privilege LLM Agents," arXiv:2606.13884, Jun. 2026.
[30] S. Sahoo, "The Controllability Trap: A Governance Framework for Military AI Agents," in *ICLR 2026 Workshop on Agents in the Wild*, arXiv:2603.03515, Mar. 2026.
[31] A. Rundgren, B. Jordan, and S. Erdtman, "JSON Canonicalization Scheme (JCS)," IETF RFC 8785, Jun. 2020.
[32] J. L. Fleiss, "Measuring Nominal Scale Agreement Among Many Raters," *Psychological Bulletin*, vol. 76, no. 5, pp. 378–382, 1971.
[33] K. Krippendorff, *Content Analysis: An Introduction to Its Methodology*, 4th ed., SAGE, 2018.
[34] K. L. Gwet, "Computing Inter-Rater Reliability and Its Variance in the Presence of High Agreement," *British Journal of Mathematical and Statistical Psychology*, vol. 61, no. 1, pp. 29–48, 2008.
[35] L. Sha, "Using Simplicity to Control Complexity," *IEEE Software*, vol. 18, no. 4, pp. 20–28, Jul. 2001, doi: 10.1109/MS.2001.936213.
[36] ASTM F3269-21, *Standard Practice for Methods to Safely Bound Behavior of Aircraft Systems Containing Complex Functions Using Run-Time Assurance*, ASTM International, 2021, doi: 10.1520/F3269-21.
[37] IEC 80601-2-77:2019+AMD1:2023 (Ed. 1.1, consolidated), *Medical electrical equipment — Part 2-77: Particular requirements for the basic safety and essential performance of robotically assisted surgical equipment*, IEC, 2023.
[38] ISO 14971:2019, *Medical devices — Application of risk management to medical devices*, ISO, 2019 (confirmed 2025).
[39] IEC 62304:2006+AMD1:2015 (consolidated), *Medical device software — Software life cycle processes*, IEC, 2015.
[40] P. Kazanzides, Z. Chen, A. Deguet, G. S. Fischer, R. H. Taylor, and S. P. DiMaio, "An open-source research kit for the da Vinci Surgical System," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2014, pp. 6434–6439, doi: 10.1109/ICRA.2014.6907809; dVRK software and ROS 2 interfaces, github.com/jhu-dvrk (research software, not a clinical system).
[41] A. Dal Pozzolo, O. Caelen, R. A. Johnson, and G. Bontempi, "Calibrating probability with undersampling for unbalanced classification," in *Proc. IEEE Symp. Ser. Comput. Intell. (SSCI)*, Cape Town, 2015, pp. 159–166, doi: 10.1109/SSCI.2015.33 (the ULB/Worldline credit-card fraud dataset, 284,807 transactions, 492 fraud-labeled).
[42] D. D. Clark and D. R. Wilson, "A comparison of commercial and military computer security policies," in *Proc. IEEE Symp. Security and Privacy*, Oakland, CA, USA, 1987, pp. 184–194.
[43] W. J. Fokkink and P. R. Hollingshead, "Verification of interlockings: From control tables to ladder logic diagrams," in *Proc. 3rd Int. Workshop on Formal Methods for Industrial Critical Systems (FMICS)*, 1998, pp. 171–185.
[44] Intuitive Surgical, Inc., "Intuitive shares vision for an AI-enabled future of healthcare," press release, Sunnyvale, CA, USA, Jul. 23, 2026. [Online]. Available: https://isrg.intuitive.com/news-releases/news-release-details/intuitive-shares-vision-ai-enabled-future-healthcare (accessed Sep. 9, 2026).

Abhinandan Gill-Lakhowal (Member, IEEE) received the B.Tech. degree in engineering and the M.Eng. degree from the University of British Columbia, Vancouver, BC, Canada. He has more than ten years of enterprise product-delivery experience across regulated industries, including financial services, aviation, and higher education. He is currently the Principal Advisor of Gillian Holdings Incorporated, Calgary, AB, Canada, and an independent AI governance researcher and strategist. He is the author of the L-DREA deterministic runtime enforcement architecture, its companion concurrence theory, and the published enforcement-mechanism patent application cited herein [15]– [17].